

Tests That Earn Their Keep

A test suite is not free and it is not automatically an asset. It runs on the trust engineers place in it, and a bad one is worse than none at all. Here is what a suite is for, how to classify it, how to write tests that never need touching, what to do when the real dependency will not fit — and what none of it can tell you.

1

What a suite is for

Two purposes, two axes, the pyramid, and the arithmetic of flakiness.

2

Tests that never need changing

Brittleness and unclarity: public APIs, state over interactions, behaviours over methods.

3

Standing in for the real thing

Seams, fakes, stubbing, interaction testing — and why the real implementation wins.

4

What small tests cannot tell you

The five gaps, fidelity, and the structure and cost of a larger test.

Print double-sided, short-edge bind, A4 · 100% scale (no “fit to page”).

Answer key starts at Section 9 — keep a sheet of paper over it.

How to use this booklet

The loop, per section

- 1. Read the plain-version box first. If it doesn't land, the rest won't either.
- 2. Read the teaching. Say each boundary out loud: “X is *this*, its look-alike Y is *that*.”
- 3. Cover the page. Write the answer to **Q1** in the blank lines, in your own words, in full sentences. **Then** check the key.
- 4. Score yourself on completeness. Then do **Q2** and note whether it beat Q1.
- 5. Only then move on.

THE RULE THAT DOES MOST OF THE WORK

If you find yourself thinking “yes, I know this” — that is *recognition*, and it is not learning. The only evidence that counts is a written answer produced with the page covered.

WHAT THIS SUBJECT ASKS OF YOU THAT OTHERS DON'T

Nearly everybody agrees that testing is good, which is exactly what makes this material hard to learn properly: the agreement hides the fact that most of it is a set of **trade-offs with stated prices**. A test that is easier to write is usually harder to maintain. A double that makes a test fast makes it less truthful. A test that covers more of the system tells you less about where the fault is. So read for the *price* attached to each recommendation, not for the recommendation. Two habits keep you honest. First, for every practice, ask **which specific failure it prevents and what it costs in exchange** — if you cannot name both, you have absorbed a slogan. Second, notice how much of this subject is about the *test suite as a thing that can itself go wrong*: it can be slow, untrustworthy, brittle, unowned, or quietly measuring nothing. The central claim of the whole subject is that a suite in that state is not neutral. It is a liability.

The spacing schedule

Do a part, then review it after **1 day, 3 days, 7 days, 16 days** and **35 days**. A review is not rereading: it is covering the page and answering the questions in Section 10's tracker. On a fail, that row goes back to a 1-day interval.

What's in here

SECTION	CONTENTS
1	Learning goal, the core model in one paragraph, and the four-part roadmap.
2–5	One part each, always in the same order: plain version → everyday analogy → term table with boundaries → trade-offs → concept-boundary box → anchor sketch → plain/expert phrasing → two written questions on cases that are not in the teaching → a 2×2 → a carry-out rule.
6	The whole subject as one concept sketch, with a question on every node, plus the master 2×2.
7	Symptom → diagnosis → move, and the reverse mapping: you see a practice, what is it for?
8	Numbers worth remembering, and the glossary.
9	Answer key for the eight written questions.
10	Spaced-review tracker and the final quiz.

Learning goal & roadmap

LEARNING GOAL — WHAT YOU SHOULD BE ABLE TO DO WHEN YOU CLOSE THIS

Faced with a test suite to design, inherit, or repair, you can:

1. **Classify any test on both axes** — the resources it is allowed to consume and the amount of code it validates — and say why the two are distinct and which one the constraints are actually enforcing.
2. **Diagnose a suite that is costing more than it returns**, naming whether the problem is brittleness, unclarity, slowness or flakiness, and give the specific remedy and its price.
3. **Choose between a real dependency and each kind of stand-in**, and say what each one buys, what it costs in truthfulness, and the condition that makes it the wrong choice.
4. **Say what unit tests structurally cannot tell you**, name the gaps a larger test exists to close, and describe the four phases and the two competing properties of the thing being tested.

The core model in one paragraph

Start from what a test is: **one behaviour, one specific input, an observable output, and a controlled environment**. Thousands of those together tell you how well a product conforms to its intended design and, more importantly, when it does not. But the suite is not a free asset, because **tests derive their value from the trust engineers place in them** — if testing becomes a productivity sink that constantly induces toil and uncertainty, engineers lose trust and find workarounds, and **a bad test suite can be worse than no test suite at all**. Everything else follows from protecting that trust. You classify tests on **two distinct axes** — *size*, the resources a test may consume, and *scope*, how much code it validates — and enforce the smallest size available, because speed and determinism are what keep the suite runnable. You aim for a mix weighted heavily toward narrow tests, because they are fast, stable, and pinpoint the fault. Then you write those tests so they **never need to be touched again** unless the system's requirements change, which means testing through public interfaces and asserting on resulting state rather than on the sequence of calls that produced it. Where the real dependency will not fit inside those constraints you substitute something — but each substitution trades **fidelity**, and a stand-in that diverges from the thing it replaces produces tests that pass while the system is broken. And finally, none of this can tell you about configuration, load, emergent behaviour, or the unanticipated: **a unit test is ensconced in a vacuum**, which is exactly what makes it fast and exactly what makes larger tests necessary.

TENTH-GRADER VERSION — THE WHOLE THING AT ONCE

- A test gives the program a known input and checks what comes out. Thousands of them tell you whether the thing still works.
- Tests are only worth anything if people believe them. Once people stop trusting the tests, the tests are worse than useless — they cost time and change nothing.
- So keep them fast and keep them reliable. That's why the small ones are restricted to one process and aren't allowed to touch the network.
- Two different questions: how much is this test allowed to *use*, and how much of the code is it *checking*? They're not the same question.
- Write tests so that changing how the code works doesn't break them — only changing what it *does* should.
- Check what the system ended up like, not the exact steps it took to get there.
- Sometimes the real database won't fit in a test, so you use a stand-in. But a stand-in that behaves differently gives you a green test and a broken system.
- And small tests live in a vacuum. They can't tell you about bad configuration, heavy traffic, or anything nobody thought of.

The four parts, and why they're in this order

PART	WHAT IT COVERS	WHY HERE
1 What a suite is for	The two purposes of testing; why a suite runs on trust; size and scope as two distinct axes; the constraints on each size; the recommended mix and its two antipatterns; the properties every test should have; the arithmetic of flaky tests; what to test and why coverage is a poor answer; and the pitfalls a large suite develops.	First because it supplies the standard everything else is measured against. Every later practice is an answer to “what keeps this suite fast, trusted and worth running?”
2 Tests that never need changing	Maintainability, and its two failure modes — brittle and unclear; the four kinds of change to production code and which one should touch tests; testing through public interfaces; asserting on state rather than interactions; completeness and conciseness; behaviours rather than methods; naming and structure; keeping logic out of tests; failure messages; and preferring descriptive duplication to deduplication.	Second, because this is where most of a suite's cost is actually incurred. A test is written once and paid for every time somebody changes the code near it.
3 Standing in for the real thing	The three impacts of substitution; seams and dependency injection; the three techniques — faking, stubbing, interaction testing; why real implementations are preferred and when they are not; what makes a fake trustworthy and who owns it; the three dangers of overused stubbing; and when validating calls rather than results is warranted.	Third, because Part 1's size constraints force the question and Part 2's brittleness rules decide the answer. Every technique here is a trade of truthfulness for speed.
4 What small tests cannot tell you	Fidelity; the five gaps unit tests leave; the three developer-friendly properties that larger tests violate; ownership and standardisation; the four phases of a larger test; the forms the system under test can take and the two properties that conflict; seeded data and traffic; the three ways to verify; the catalogue of larger test types; and how to keep them fast, unflaky, understandable and owned.	Last, because it is defined by the limits of everything before it — and because its central quantity, fidelity, is the price Part 3 was paying without naming it.

“A test should be obvious upon inspection.”

What a suite is for

Two purposes, the trust it runs on, the two axes it is classified by, and the arithmetic that decides whether it survives.

TENTH-GRADER VERSION

- Tests catch bugs. But the bigger reason to have them is that they let you *change* things without being frightened.
- They only work if people believe them. A suite nobody trusts costs time and prevents nothing.
- Two separate questions about any test: what is it allowed to use (one process? one machine? anything?), and how much code is it actually checking?
- The small ones can't sleep, can't touch the disk, can't touch the network. That's not fussiness — it's what stops them being slow and unpredictable.
- Mostly write small, narrow ones. A few medium. Very few enormous. Upside-down versions of that mix have names, and both are bad.
- A test that fails randomly one time in a thousand sounds fine until you run ten thousand tests a day.
- Test anything you don't want to break. And don't judge a suite by what percentage of lines it runs — running a line isn't checking it.

THE EVERYDAY VERSION

A bakery installs an alarm that sounds when an oven runs too hot. For a while it is the most valuable thing in the building: nobody has to stand and watch, and a batch is never lost. Then the alarm starts going off when a door slams, and once or twice for no reason anybody can find. The staff investigate the first few. By the fourth week they glance at it and carry on. By the sixth somebody puts tape over the speaker. Notice what has happened: **the alarm still works exactly as well as it did on the first day, and it is now worth less than nothing** — it costs money to maintain and it no longer protects anything, and worse, the staff have stopped watching the ovens themselves because they think it is covered. That is the whole economics of a test suite in one object. It is also why the small tests are kept on such a short leash: an alarm that cannot be triggered by a slamming door is not a fussier alarm, it is the only kind that survives contact with a busy kitchen.

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
The simplest test	Four things: a single behaviour you are testing, usually a method or interface you call; a specific input ; an observable output or behaviour ; and a controlled environment such as a single isolated process.	Taken in aggregate, hundreds or thousands of such tests — a test suite — tell you how well your product conforms to its intended design and, more important, when it doesn't .	vs a script that exercises the program. The fourth element is the one people drop, and it is the one the rest of this part is about: without a controlled environment you have an observation, not a test.

The two purposes	Catching bugs is only part of the motivation. An equally important reason to test is to support the ability to change.	The later in the development cycle a bug is caught, the more expensive it is — exponentially so in many cases. But whether you are adding features, refactoring for code health, or undertaking a larger redesign, tests catch mistakes quickly and make it possible to change software with confidence. If you have a robust testing practice you need not fear change; you can embrace it.	vs quality assurance. The consequence is directional: the more and faster you want to change your systems, the more you need a fast way to test them. Writing tests also improves design — as the first client of your code, a test tells you whether the system is too tightly coupled to a database, whether the interface supports the required use cases, and whether the edge cases are handled.
The trust economy	Creating and maintaining a healthy suite takes real effort — and tests derive their value from the trust engineers place in them.	As a codebase grows so does its suite, and it will begin to face instability and slowness. If testing becomes a productivity sink, constantly inducing toil and uncertainty, engineers will lose trust and begin to find workarounds.	This is the load-bearing sentence of the whole subject: a bad test suite can be worse than no test suite at all. It is why speed, determinism and clarity are treated as first-order requirements rather than niceties — each of them is a defence of the trust the suite runs on.
Size	The resources required to run a test case — memory, processes, time — and what it is allowed to do. Not its number of lines.	Small tests run in a single process (often a single thread), and may not sleep, perform I/O, or make any other blocking call — so no network and no disk, and no third-party program such as a database. Medium tests may span multiple processes, use threads and make blocking calls including network calls to the local machine, but may not reach any other machine. Large tests remove that restriction and may span machines.	vs the traditional unit/integration split, and the distinction is deliberate: the most important qualities wanted from a suite are speed and determinism regardless of scope , and small tests are almost always faster and more deterministic. The constraints are not pedantry — they provide a sandbox that prevents engineers from shooting themselves in the foot , and in some cases the infrastructure enforces them.
Scope	How much code is being validated by a given test — which is different from how much code gets executed.	Narrow -scoped tests, commonly called unit tests, validate the logic in a small focused part of the codebase such as one class or method. Medium -scoped tests verify interactions between a small number of components — a server and its database. Broad -scoped tests validate the interaction of several distinct parts, or emergent behaviours that are not expressed in any single class or method.	vs size, with which it correlates but is not identical. When a unit test is called narrowly scoped, that refers to the code being validated, not the code being executed — a class naturally invokes its dependencies while under test. Both mixed cases exist: a broad-scoped test of an endpoint can be <i>small</i> if doubles stand in for everything out of process, and a narrow-scoped test of one method can be <i>medium</i> if validating a single path requires running an entire browser.

The mix, and its two antipatterns	A rough guideline: about 80% narrow-scoped unit tests validating the majority of business logic, 15% medium-scoped integration tests validating interactions between components, and 5% end-to-end tests validating the whole system.	Unit tests form the base because they are fast and stable, they narrow the scope and reduce the cognitive load of identifying all possible behaviours, and they make failure diagnosis quick and painless. The ice cream cone is many end-to-end tests and few of the others — slow, unreliable, hard to work with, and typical of prototypes rushed to production without addressing testing debt. The hourglass is many end-to-end and many unit tests but few integration tests, which is not quite as bad but still produces end-to-end failures that could have been caught more easily; it appears when tight coupling makes it difficult to instantiate individual dependencies in isolation .	vs a rule. Every team's mix will be a little different, and the numbers are by test case count. The reasoning behind the shape is worth keeping: favouring unit tests gives high confidence quickly and early, and larger tests act as sanity checks — they should not be viewed as a primary method for catching bugs . But unit tests cannot verify the interactions between components, such as a contract between two systems built by different teams.
Properties every test should have	Hermetic — the test contains all of the information necessary to set up, execute and tear down its environment, and assumes as little as possible about the outside world, including the order in which tests run.	They should not rely on a shared database. And a test should contain only the information required to exercise the behaviour in question : keeping tests clear and simple helps reviewers verify the code does what it says, and helps whoever must diagnose the failure. Hence the maxim: a test should be obvious upon inspection .	The reason for that maxim is structural — because there are no tests for the tests themselves, they require manual review as an important check on correctness . As a corollary, control flow statements such as conditionals and loops in a test are strongly discouraged : more complex flows risk containing bugs of their own and make the cause of a failure harder to determine.
The arithmetic of flakiness	A flaky test fails nondeterministically. At scale, tiny per-test flakiness becomes a daily tax.	If each test has even a 0.1% chance of failing when it should not , and you run 10,000 tests per day , you will be investigating 10 flakes per day — each investigation taking time from something more productive. Rerunning failures automatically trades processor cycles for engineering time, which makes sense at low levels, but rerunning a test only delays the need to address the root cause .	The real cost is not the investigations: it is that experience suggests the tests begin to lose value as flakiness approaches 1% , because it does not take many flakes before a team stops trusting the suite — and once engineers stop reacting to failures, any value the suite provided is eliminated. Most flakes come from nondeterminism in the tests themselves: clock time, thread scheduling, network latency, and lower-level concerns still.

What to test	The straightforward answer: test everything that you don't want to break . If you want confidence that a system exhibits a particular behaviour, the only way to be sure is to write an automated test for it.	That includes the usual suspects — performance, behavioural correctness, accessibility and security — and the less obvious ones, such as how a system handles failure . Rather than waiting for a catastrophe to find out how the system responds to one, simulate the common kinds: exceptions and errors in unit tests, injected remote-call errors or latency in larger ones, and disruptions to the real production network.	The rule has a sharp organisational edge, usually invoked by teams making changes across an entire codebase: if unrelated infrastructure changes pass all of your tests but still break your product, you are on the hook for fixing it and adding the additional tests . Stated as an aphorism: if you liked it, you should have put a test on it.
Why coverage is a poor answer	Code coverage measures which lines of feature code are exercised by which tests. Run 90 of 100 lines and you have 90% coverage.	It is often held up as the gold standard for test quality, and that is unfortunate: it is possible to exercise a lot of lines with a few tests, never checking that each line is doing anything useful — coverage measures only that a line was invoked, not what happened as a result. The recommendation is to measure it from small tests only, to avoid the coverage inflation that executing larger tests produces.	The more insidious problem is that, like other metrics, it quickly becomes a goal unto itself : a team sets a bar of, say, 80%, and in practice engineers treat the floor as a ceiling — why do more work than the metric requires? The better question set is about behaviours: are you confident everything customers expect to work will work; can you catch breaking changes in your dependencies; are your tests stable? Answering “do we have enough tests?” with a single number ignores a lot of context .
The pitfalls of a large suite	Two ways a growing suite turns against you: it becomes brittle , and it becomes slow .	Brittle tests over-specify expected outcomes or rely on extensive and complicated boilerplate, so they fail even when unrelated changes are made — a five-line change breaking dozens of unrelated tests. Over time that friction makes a team reticent to perform the refactoring that keeps a codebase healthy. Slow suites are run less often and so provide less benefit; parallel execution and faster hardware are eventually swamped by a large number of individually slow cases.	Two named causes worth carrying. Some of the worst brittleness comes from the misuse of mock objects — treated in Part 3. And a specific slowness trap: calls to sleep and timeout functions used as naive heuristics before checking a nondeterministic result. Half a second here or there looks harmless, but embedded in a widely used utility it adds minutes of idle time to every run . The better solution is to actively poll for the state transition at a frequency closer to microseconds , combined with a timeout in case the state is never reached.
Living with a large suite	The secret is to treat it with respect — and specifically, to treat your tests like production code .	Incentivise engineers to care about their tests, and reward them as much for having rock-solid tests as for a great feature launch . Set performance goals and refactor slow or marginal tests. When simple changes begin taking nontrivial time, spend effort making the tests less brittle. And invest in infrastructure — linters, documentation and other assistance that makes it more difficult to write bad tests .	vs discipline. The stated failure mode is economic, not moral: engineers who encounter unbearable tests find ways around the slowdown, some going as far as skipping tests entirely when submitting changes. That is risky and should be discouraged — but if a suite is causing more harm than good, eventually engineers will get their job done, tests or no tests . Reducing the number of frameworks and tools you must support increases the return on every hour spent improving them.

The limits of automation	Automated testing is not suitable for all testing tasks .	Judging the quality of search results involves human judgement; so does capturing the nuances of audio and video quality. And humans are better than automated systems at searching for complex security vulnerabilities . The general form is exploratory testing : a fundamentally creative endeavour in which someone treats the application as a puzzle to be broken, perhaps by executing an unexpected set of steps or inserting unexpected data — the specific problems to be found are unknown at the start and are gradually uncovered by probing overlooked code paths and unusual responses.	vs an argument for manual testing. The relationship is a pipeline: as soon as an exploratory test discovers an issue, an automated test should be added to prevent future regressions , and a flaw found by a human can then be run continuously and at scale. Automating the well-understood behaviours is what frees the expensive and qualitative human effort for the parts where it adds most — and avoids boring the testers to tears.
---------------------------------	--	--	---

CONCEPT BOUNDARY — THE SUITE IS AN ASSET ONLY WHILE IT IS BELIEVED

What it sounds like: tests are a safety net, so more of them is better, and a slow or noisy suite is at worst a mild inconvenience — still better than nothing.

What it actually is: a claim that the suite's entire value is mediated by human behaviour. Its value is realised only when somebody reads a failure and acts on it. That makes trust the currency, and trust is destroyed by exactly the things this part constrains: nondeterminism, slowness, brittleness and obscurity. Hence the sentence that anchors the subject — a bad suite can be worse than no suite at all — which is not rhetoric but arithmetic, since a suite people have stopped believing costs maintenance, costs investigation time, and has additionally stopped anyone from watching the system themselves.

Why that matters practically: it tells you what the size constraints are really for. Forbidding a small test from sleeping, touching disk or reaching the network is not about purity. Each prohibition removes a source of slowness or nondeterminism, which are the two things that erode belief fastest — and the reason larger sizes are permitted at all is that some risks cannot be covered any other way, at a price you should pay knowingly.

The trap: measuring the suite with a number that ignores belief. Coverage is the classic case: it records that a line ran, not that anything checked it, and the moment it becomes a target the floor turns into a ceiling. The questions that actually assess a suite are about behaviours and confidence — will the things customers rely on keep working, will you catch a breaking change in a dependency, and do people still act on a red result.

Anchor sketch — the two axes, and what keeps a suite believed

Legend: bold = a named class or rule

reversed = the fact everything else answers

Q = cover the page and answer this

A TEST = one BEHAVIOUR . one specific INPUT . an observable OUTPUT . a CONTROLLED ENVIRONMENT.
 thousands of them say how well the product matches its design and -- more important -- WHEN IT DOES NOT.

TWO PURPOSES catch bugs (cost rises, often exponentially, the later they are caught) AND SUPPORT THE ABILITY TO CHANGE.
 -> the more and faster you want to change, the more

you need a fast way to test.

-> the test is the FIRST CLIENT of your code: too coupled to a database? does the API fit the use cases? are the edge cases handled?

|

TESTS DERIVE THEIR VALUE FROM THE TRUST ENGINEERS

PLACE IN THEM. A BAD SUITE IS WORSE THAN NO SUITE.

a productivity sink breeds workarounds.

everything below is a defence of that trust.

TWO DISTINCT AXES

SIZE = the RESOURCES it may use. not lines of code.

SMALL one process, often one thread. NO sleeping,
NO I/O, NO blocking calls -- so no network,
no disk, no database.

MEDIUM several processes, threads, blocking calls,
the local machine only.

LARGE any machine. the fewest guarantees.

-> why not "unit vs integration"? because SPEED AND
DETERMINISM are wanted REGARDLESS OF SCOPE.

-> the constraints are a SANDBOX that stops you
shooting yourself in the foot.

SCOPE = how much code is VALIDATED (not executed).

NARROW one class or method . MEDIUM interactions
between a few components . BROAD several parts, or
EMERGENT behaviour no single method expresses.

-> they correlate but come apart: a BROAD test can
be SMALL (doubles for everything out of process),
and a NARROW test can be MEDIUM (one code path
that needs a whole browser).

Q which axis do the enforced constraints police?

THE MIX ~80% narrow . ~15% medium . ~5% end-to-end

unit tests are the base: fast, stable, and they
NARROW the search when something breaks.

larger tests are SANITY CHECKS -- not the primary
way to catch bugs.

but unit tests CANNOT verify a contract between two
systems built by different teams.

ICE CREAM CONE many end-to-end, few others. slow,
unreliable. the prototype rushed to production.

HOURLASS many end-to-end + many unit, few in the
middle. caused by TIGHT COUPLING -- dependencies
cannot be instantiated in isolation.

EVERY TEST **HERMETIC** (carries its own setup, execution
and teardown; assumes nothing about ordering; no
shared database) and OBVIOUS UPON INSPECTION.

-> WHY? THERE ARE NO TESTS FOR THE TESTS. review is
the only check on their correctness.

-> hence: no conditionals, no loops.

FLAKINESS IS ARITHMETIC . $0.1\% \times 10,000$ tests/day

= 10 investigations a day. AT ~1% THE TESTS BEGIN
TO LOSE VALUE -- people stop reacting to failures.
reruns trade CPU for engineer time and only DELAY
the root cause.

TEST EVERYTHING YOU DO NOT WANT TO BREAK -- including
performance, accessibility, security, and HOW IT
FAILS. if someone else's change passes your tests

and breaks your product, THAT IS ON YOU.

COVERAGE IS A WEAK ANSWER . it records that a line RAN, not that anything CHECKED it. and as a target, the floor becomes a CEILING.

-> measure it from SMALL tests only.

HOW A BIG SUITE TURNS ON YOU

BRITTLE -- over-specified or boilerplate-heavy, so UNRELATED changes break it. teams then stop refactoring.

SLOW -- run less often, worth less. the classic cause: sleep() as a "wait and check" heuristic, buried in a shared utility.

-> POLL for the transition at microsecond frequency, with a timeout.

-> the cure is cultural: TREAT TESTS LIKE PRODUCTION CODE. reward rock-solid tests like launches. build linters and docs that make BAD tests hard.

AND THE EDGE OF AUTOMATION . human judgement for quality, for audio and video, for finding complex security flaws. EXPLORATORY TESTING treats the product as a puzzle to break, with the target UNKNOWN at the start.

-> whatever it finds, AUTOMATE IT afterwards.

- Q Define size and scope, and give a case where each comes apart from the other.
- Q Give the flakiness arithmetic and the threshold at which a suite stops being believed.
- Q Why is “no conditionals or loops in a test” a consequence rather than a style rule?

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
Constraining small tests — no sleeping, no I/O, no blocking calls	Speed and determinism close to what the processor can manage, and a sandbox that makes it hard to write a slow or flaky test by accident.	Anything genuinely needing a database, a network or a browser must be doubled or promoted to a larger size.	Whenever possible, regardless of the test's scope. This is what keeps the whole suite runnable.
Writing a larger test instead	Coverage of risks smaller tests cannot reach, and behaviour closer to how the system really runs.	Slower, likelier to be nondeterministic, and further from a clean pass/fail signal.	Large tests are <i>mostly</i> reserved for full-system checks that are more about validating configuration than code, and for legacy components where doubles are impossible.
Rerunning a failing test automatically	Fewer interruptions now — processor cycles traded for engineering time.	It only postpones the root cause, and the underlying flakiness keeps compounding.	At low levels of flakiness, as a stopgap. Never as the answer.
Setting a coverage target	A visible number, and some insight into which code is untested.	Engineers treat the floor as a ceiling, and the number certifies lines that were run but never checked.	As a diagnostic on small tests only — not as a definition of “enough tests”.

Testing behaviour you merely hope is stable	Nothing — that is the point. An untested behaviour is one you have no claim on.	Every test is code to maintain.	Test everything you do not want to break, including failure handling. Anything else is an unwritten expectation.
Polling for a state transition instead of sleeping	Test time measured in the microseconds it actually takes rather than the half-second somebody guessed.	More code than a sleep, and you need a timeout for the case where the state never arrives.	Always, and especially in shared utilities, where one careless sleep multiplies across every run.
Exploratory testing by a human	The unanticipated — the paths and inputs nobody thought to specify, including subtle security flaws.	It does not scale, and it finds nothing twice.	For new and launched systems alike, provided every finding is converted into an automated test afterwards.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO <i>UNDERSTAND IT</i>	SAY IT THIS WAY TO SOUND LIKE AN <i>EXPERT</i>
“We’ve got tests, so we’re covered.”	“Does anyone still act on a red build? A suite nobody believes is worse than none.”
“It’s a unit test, so it’s small.”	“Those are two different axes. Scope is what it validates; size is what it’s allowed to consume.”
“Why can’t this small test hit the database?”	“Because I/O and blocking calls are the two biggest sources of slowness and nondeterminism. The constraint is the sandbox.”
“It only fails occasionally.”	“At 0.1% across ten thousand runs that’s ten investigations a day — and near 1% people stop reading the results at all.”
“We’ll just retry it in CI.”	“That trades compute for engineer time, which is fine — but it postpones the root cause rather than fixing it.”
“We’re at 82% coverage.”	“Coverage says a line ran, not that anything checked it. What behaviours are we confident about?”
“Most of our tests are end-to-end.”	“That’s an ice cream cone. Slow, unreliable, and it means failure diagnosis starts from the whole system.”
“We have plenty of unit and end-to-end tests.”	“And nothing in between — an hourglass. That usually means coupling stops us instantiating dependencies alone.”
“I added a loop to the test to cover both cases.”	“Nothing tests the test. Logic in a test is a bug waiting to hide — write two tests.”
“Their change broke our product but our tests passed.”	“Then the behaviour wasn’t tested, and adding that test is on us.”

PART 1 · QUESTION 1

A payments team's suite has 4,000 tests. Most exercise the whole service through its HTTP surface with a real database container and a real message broker; the team is proud that this catches “real” problems. The suite takes 55 minutes and is run nightly rather than per change. Roughly forty tests fail on any given night for reasons nobody can reproduce, so the team's rule is to rerun the build twice and merge if it goes green either time. Coverage sits at 84% against a target of 80% and has not moved in a year. When a shared library team changed a date-parsing helper last month, the payments service began rejecting valid transactions in production; the payments team's tests all passed, and they have asked the library team to be more careful.

(a) Classify the typical test on both axes and name the suite's shape, giving the two properties that shape is known for. **(b)** Work out what the nightly failures imply about trust, using the stated arithmetic and threshold, and assess the rerun rule. **(c)** The coverage figure is being read as evidence of health. Give the two separate reasons it is weak evidence, and state what should be asked instead. **(d)** Assign responsibility for the date-parsing incident under the stated rule, and say what the payments team owes as a result.

PART 1 · QUESTION 2

A team is splitting a monolith into services and rewriting its tests. A helper called `waitForReady()`, used by roughly 600 tests, sleeps for 400 milliseconds before checking a status flag. A new test covers one method that converts a timestamp into a customer's local calendar date; the conversion table is a 200 MB regional-offset file the method loads from disk, so the test cannot avoid I/O. A reviewer objects that “it tests a single method, so it has to be small”. A second new test drives the whole nightly-report pipeline — ingest, aggregate, format — from top to bottom, but every collaborator is an in-process stand-in and the run finishes in 90 milliseconds inside one process. The same reviewer objects that it “covers far too much to be small”. Separately, a security researcher found a flaw last quarter by feeding the login form deliberately malformed sequences nobody had considered; the team's response was to fix the bug.

(a) Compute the cost of the sleeping helper across the suite, name the antipattern, and give the recommended replacement including its safeguard. **(b)** Adjudicate both of the reviewer's objections, using the correct definitions, and say what general point the pair of cases illustrates. **(c)** Name the kind of testing the researcher performed, give its defining property, and state the step the team omitted. **(d)** The team wants a target mix for the rewritten suite. Give it with the reasoning for the shape, and name the one thing the largest tests should not be relied on for.

2x2 — is it fast and deterministic, and does it tell you where the fault is

COLUMNS: DOES A FAILURE LOCALISE THE FAULT? →

	Yes — narrow scope, so the failure names the culprit	No — broad scope, so a failure means “somewhere in there”
FAST and DETERMINISTIC (small)	The base of the suite Runs in a single process, never sleeps or blocks, and when it goes red you know roughly which class did it. This is why the mix is weighted here. <i>Move:</i> write the smallest test the behaviour allows, whatever its scope — and keep it hermetic and free of control flow, because nothing tests the test.	Fast, and vague when it breaks A broad-scoped test kept small by doubling everything out of process. Legitimate and useful — it just cannot tell you where the fault is. <i>Move:</i> keep it, and be aware you have traded diagnosis for reach. Where the doubles stand in for something real, Part 3’s fidelity question is now live.
SLOW or FLAKY (medium/large)	Narrow, and expensive anyway One code path that happens to need a browser, a process boundary or a real dependency. Small in what it validates, medium or large in what it consumes. <i>Move:</i> accept the size rather than faking the classification, and isolate these from the fast suite so they do not slow the everyday workflow.	Where suites go to die Slow, nondeterministic, and uninformative when red. Individually defensible; as the bulk of a suite it is the ice cream cone. <i>Move:</i> this is the cell trust leaks out of. Push behaviour down into smaller tests, reserve these for whole-system checks and legacy components, and treat every flake as a defect rather than a rerun.

Read it as: the row is *size* and the column is *scope*, drawn deliberately as two axes because the industry habit of collapsing them into one word is what produces the wrong argument in review. The two off-diagonal cells are the proof that they are independent: a broad test can be fast, and a narrow test can be expensive, and mistaking either for a classification error leads people to write the wrong test.

CARRY-OUT RULE FROM THIS PART

Judge a suite by whether people still act on a red result — and write the smallest test the behaviour allows, whatever its scope, because speed and determinism are what keep that true.

Tests that never need changing

The two ways a test turns into a liability, and the practices that stop a suite becoming the thing that resists change.

TENTH-GRADER VERSION

- The ideal test is one you write once and never touch again — unless what the system is *supposed* to do changes.
- Two things go wrong. A test breaks when nothing was actually broken. Or a test fails and nobody can work out what it was ever for.
- Test through the front door — the same way real users call the code. If you test the private bits, every tidy-up breaks a test.
- Check what the system ended up like, not the exact list of calls it made along the way. There's more than one way to get the right answer.
- Write one test per *behaviour*, not one per method. Methods do several things; behaviours don't.
- Name the test after the behaviour. If you need the word "and" in the name, it's two tests.
- No loops or if-statements in tests. Nothing checks the test, so it has to be obviously right just by looking.
- A bit of repetition is fine in tests. Clever sharing that hides the details is not.

THE EVERYDAY VERSION

A landlord wants to know that a flat's front door still locks. There are two ways to check. The first: turn the key from the outside and try the handle. The second: take the lock apart, note that the third pin sits at 4.1 millimetres and the cam rotates 38 degrees, and assert those numbers. Both confirm the door is locked today. But replace the lock with a better one, or have the locksmith adjust the mechanism, and the second check screams while the door is as secure as ever — **and the landlord, who has been woken at 3 a.m. by that alarm four times, eventually disconnects it.**

Notice the second failure too: two years on, nobody can remember why 38 degrees was the number, or whether it still should be. The first check has neither problem. It survives any change to the mechanism and fails only when the door genuinely does not lock, and anybody reading it can see in one line what it was for.

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
Maintainability	Maintainable tests are ones that “just work” : after writing them, engineers do not need to think about them again until they fail, and those failures indicate real bugs with clear causes.	The failure it prevents is concrete: a small feature change comes back with a screenful of test failures, none of which is a real bug — each one broke an assumption a test made about the internal structure of the code. The day goes on repairs, the hacks used to fix them make the tests harder to understand next time, and what should have been a quick job takes hours.	vs correctness. Tests are cheap to write and expensive to own, and in that scenario testing had the opposite of its intended effect, draining productivity rather than improving it while not meaningfully increasing the quality of the code under test. Two categories cover the damage: the tests were brittle , and the tests were unclear . Everything in this part attacks one or the other.

Brittleness	A brittle test fails in the face of an unrelated change to production code that does not introduce any real bug.	In a small codebase with a few engineers, tweaking a few tests per change may not matter. But a team that regularly writes brittle tests will find maintenance consuming a larger and larger share of its time — and if a set of tests must be manually tweaked by engineers for each change, calling it an “automated test suite” is a bit of a stretch.	vs a flaky test, which fails nondeterministically without any change to production code. Both destroy trust; they have different causes and different cures. At scale the cost compounds: a single sweeping change can trigger hundreds of thousands of tests, so spurious breakage in even a small percentage wastes enormous amounts of time.
The four kinds of change	The framework that makes “unchanging” precise. The ideal test never needs to change unless the requirements of the system under test change.	Pure refactoring — internals change, the interface does not; tests should not need to change, since their role is to confirm the refactoring did not change behaviour. New features — existing behaviours are unaffected, so you write new tests and touch no old ones. Bug fixes — the bug means a case was missing, so the fix brings the missing test with it, again touching nothing existing. Behaviour changes — the one case where existing tests are expected to change.	Each of the first three has a diagnostic edge: a test that must change during a refactoring means either the change is not a pure refactoring, or the tests were not written at an appropriate level of abstraction. The fourth is qualitatively different and much more expensive, because users rely on current behaviour: changing a test here means you are breaking an explicit contract , whereas changes in the other three mean you were relying on an unintended contract.
Test via public interfaces	By far the most important practice: invoke the system being tested in the same way its users would , calling its public surface rather than its implementation details.	The argument is airtight by construction: if tests work the same way as the system's users, a change that breaks a test might also break a user. Peering into internal state or calling methods that are not publicly exposed means almost any refactoring — renaming a method, extracting a helper class, changing a serialisation format — breaks the test, even though the change is invisible to real users.	“Public” here means the surface exposed to third parties outside the team that owns the code , which does not always match a language's visibility keywords. Defining the scope of a unit is more art than science; three rules of thumb. A class existing only to support one or two others probably should not be considered its own unit and should be tested through them; anything designed to be used without consulting its owners almost certainly constitutes a unit and should be tested directly ; and a general-purpose support library accessible only to its owners is also a unit — the resulting redundancy is valuable, since without it removing one caller could silently open a coverage gap.

State, not interactions	Two ways to verify. With state testing you observe the system itself after invoking it. With interaction testing you check that it took an expected sequence of actions on its collaborators.	Interaction tests are more brittle for the same reason testing a private method is: they check how a system arrived at its result, whereas usually you should care only what the result is. Verifying that a particular call was made against a database fails in both directions — if a bug deletes the record immediately afterwards the test still passes , and if the system is refactored to call a slightly different interface to write an equivalent record the test fails though nothing is wrong.	vs a stylistic preference. The named cause is over-reliance on mocking frameworks , which make it trivial to create doubles that record and verify every call — a strategy that leads directly to brittle interaction tests. Hence the stated preference for real objects over mocked objects, as long as the real objects are fast and deterministic. Part 3 takes the subject up properly.
Complete and concise	A test is complete when its body contains all the information a reader needs to understand how it arrives at its result, and concise when it contains no other distracting or irrelevant information.	The failing case has both defects at once: a constructor loaded with irrelevant arguments, and the values that actually matter buried in a helper, leaving the reader asking where the expected number came from. The fix pulls the meaningful inputs into the test body and pushes the irrelevant construction into a helper.	vs short. The two pull in opposite directions and both must hold — which is why the resolution is often to violate deduplication if it leads to clearer tests. The test to apply: a test's body should contain all the information needed to understand it, without containing any irrelevant or distracting information.
Behaviours, not methods	A behaviour is any guarantee a system makes about how it will respond to a series of inputs while in a particular state — expressible as <i>given, when, then</i> .	The instinct is to mirror the code's structure, one test method per production method. It is convenient at first and degrades: as the method grows more complex, so does its test. Framing tests around methods encourages unclear tests because a single method often does a few different things under the hood and might have several tricky edge cases. The mapping is many-to-many : most nontrivial methods implement multiple behaviours, and some behaviours rely on the interaction of multiple methods.	Behaviour-driven tests are clearer for three stated reasons: they read more like natural language rather than requiring laborious mental parsing; they express cause and effect more clearly because each is limited in scope; and being short and descriptive, they make it easier to see what is already tested and encourage adding new streamlined tests instead of piling onto existing ones. Structure them so the three parts are explicit, which lets a reader work at three levels — the name, then the given/when/then, then the code. The common violation is interleaving assertions among multiple calls , merging “when” and “then” so the action cannot be distinguished from the expected result.

Naming after the behaviour	A test's name should summarise the behaviour it tests: both the actions being taken on the system and the expected outcome , sometimes with the prior state.	The name matters because it is often the first or only token visible in failure reports — your best opportunity to communicate the problem when the test breaks — and the most direct expression of the test's intent. Names are far more verbose than production code would tolerate, and that is fine: you never write code that calls them, and humans read them in reports . A useful trick when stuck is to start with “should”, so that class and test name read as a sentence.	vs naming after the method under test, which is the method-oriented habit. The naming rule doubles as a scope check: if you need the word “and” in a test name, there is a good chance you are testing multiple behaviours and should be writing multiple tests . Reading all the test names in a suite should give a good sense of the behaviours the system implements.
No logic in tests	Clear tests are trivially correct upon inspection . Logic — operators, loops, conditionals — forces mental computation instead of reading the answer off the screen.	Production code must generalise over any input, so it earns its logic and has tests to check it. Test code does not have that luxury: if you feel you need to write a test to verify your test, something has gone wrong . The worked case is a single string concatenation building an expected URL, which quietly conceals a doubled separator; writing the expected value out in full makes the bug immediately visible.	vs duplication being bad. The lesson is that duplicating the value was a small price to pay for making the test more descriptive and meaningful . And the risk scales: if humans are bad at spotting bugs from string concatenation, they are worse at spotting them in loops and conditionals — so stick to straight-line code over clever logic, and tolerate some duplication when it makes the test more meaningful .
Clear failure messages	The last aspect of clarity concerns not how a test is written but what an engineer sees when it fails . Ideally they could diagnose the problem from the message alone, without opening the test.	A good message contains much the same information as the test's name: the desired outcome, the actual outcome, and any relevant parameters . “Test failed: account is closed” leaves the reader unable to tell whether the closure was the cause or the unmet expectation; naming both the expected state and the actual object removes the ambiguity.	vs an assertion library's default output, which is only as informative as what you handed it. An assertion given nothing but a boolean can say only that it expected true and got false; one given the subject can report which collection failed to contain what. Where no such helper exists, it should always be possible to specify the important information manually .

Descriptive over deduplicated	Production code aims to avoid repetition so that a concept lives in one place. Test code should instead favour descriptive and meaningful phrases — a little duplication is fine so long as it makes the test simpler and clearer.	The cost/benefit differs for two reasons. Deduplication earns its keep by making change easier, but good tests are designed to be stable and you usually want them to break when the system changes — so the benefit is smaller. Meanwhile the cost of complexity is larger, because production code has a test suite to keep it honest as it grows complex, whereas tests must stand by themselves. The failing example hides its inputs and its assertions behind helpers full of logic, so the reader cannot see the case being tested — or the bug in the helper.	The two principles are complementary, not opposed: helper methods and infrastructure still make tests clearer by removing repetitive steps whose details are irrelevant to the behaviour under test. The test is the motive — refactor toward descriptive and meaningful, not solely to reduce repetition. Three specific patterns follow in the next row.
Three ways tests share code	Shared values, shared setup, and shared helpers — each useful and each with a characteristic failure.	Values: a bank of shared constants makes tests concise and makes it hard to see why a particular value was chosen; better-named constants help a little but still obscure the detail and tempt reuse where the name does not quite fit. The better technique is a helper that sets sensible defaults and lets each test specify only the values it cares about. Setup: best used to construct the object under test and its collaborators when most tests do not care about the arguments — but a test that <i>does</i> care about a particular value should state that value directly, overriding the default, rather than leaving a reader hunting for where it came from. Helpers: a general <i>validate</i> method called at the end of every test is the bad extreme — such tests are less behaviour-driven, their intent is hard to determine, and a single bug makes a large number of them fail at once.	The good version of a validation helper is narrow: the best ones assert a single conceptual fact about their inputs, and they earn their place when that fact is conceptually simple but needs looping or conditional logic that would reduce clarity inside the test body. Beyond a single suite, shared code becomes test infrastructure, which is closer to production code than to test code: it has many callers, is hard to change without breakages, must be treated as its own product, and must always have its own tests.

CONCEPT BOUNDARY — A TEST IS A CONTRACT, AND YOU CHOOSE WHICH ONE

What it sounds like: a list of style preferences about how to write tests — front door rather than back, state rather than calls, behaviours rather than methods, a bit of repetition rather than clever helpers. Agreeable, and easy to treat as taste.

What it actually is: one idea applied four times. Every test asserts a contract, and you get to choose whether that contract is the one your users actually depend on or an accidental one nobody promised. Test the private method and you have contracted the internal structure. Assert the call sequence and you have contracted the route rather than the destination. Write one test per method and you have contracted the code's shape. Each choice makes a change that breaks nothing real break a test — which is the definition of brittleness.

Why that matters practically: it gives you a diagnostic instead of a rulebook. When a test breaks during a refactoring, the question is not “which rule did we violate?” but “what did this test contract that we never promised?” And it explains why behaviour changes are the one legitimate reason to edit an existing test: there you really are breaking an explicit contract, and the cost of updating tests is small next to the cost of updating every user.

The trap: treating clarity as the softer half of the list. Clarity is what makes the contract inspectable, and it has a hard structural justification — **there are no tests for the tests**, so manual reading is the only check on their correctness. That is why logic is forbidden rather than discouraged, why a little duplication is a bargain, and why a test nobody can decipher is worse than absent: it will eventually be deleted, having provided nothing for however many years it existed.

Anchor sketch — the two failure modes, and the practices that answer them

Legend: bold = a named practice or rule

reversed = the fact everything else answers

Q = cover the page and answer this

THE IDEAL TEST NEVER CHANGES AGAIN UNLESS THE REQUIREMENTS OF THE SYSTEM UNDER TEST CHANGE.

maintainable = it "just works", and when it fails that means a REAL BUG WITH A CLEAR CAUSE.

|

TWO WAYS IT GOES WRONG

BRITTLE fails on an unrelated change that introduced NO REAL BUG. (vs FLAKY = fails randomly with NO production change at all. different cure.)
-> if every change needs manual tweaks, calling it "automated" is a stretch.

UNCLEAR it fails and nobody can tell what it was for. worse than unclear production code: you can find that out by seeing what breaks without it. remove an unclear TEST and nothing happens -- so obscure tests get DELETED, having given zero value for however many years they existed.

|

FOUR KINDS OF PRODUCTION CHANGE -- only ONE should touch existing tests

PURE REFACTORING -> no test changes. if a test changes: either it was not a pure refactoring, or the tests were at the WRONG LEVEL OF ABSTRACTION.

NEW FEATURE -> add tests, change none.

BUG FIX -> the bug proves a case was missing. add that case, change nothing.

BEHAVIOUR CHANGE -> the ONE legitimate case. and it

is much more expensive, because users rely on the old behaviour.
-> the first three break an UNINTENDED contract.
the fourth breaks an EXPLICIT one.

----- ANSWERING BRITTLINESS

TEST VIA PUBLIC APIS . call it the way its USERS do.
then a change that breaks a test MIGHT ALSO BREAK
A USER -- an explicit contract.
"public" = exposed to third parties outside the
owning team; NOT the language keyword.
which things are units? a helper for one or two
classes -> NO, test it through them. usable without
asking its owners -> YES. a general support library
-> YES (the redundancy is worth it).
TEST STATE, NOT INTERACTIONS . check what the system
IS, not the route it took.
verifying "the database was called" fails BOTH ways:
a bug deletes the record after -> STILL PASSES.
a refactor writes an equivalent record differently
-> FAILS for nothing.
cause: over-reliance on mocking frameworks.
prefer REAL objects while they are fast and
deterministic.

----- ANSWERING UNCLARITY

COMPLETE AND CONCISE . everything needed to understand
it, and nothing else. both, together.
BEHAVIOURS, NOT METHODS . a behaviour = a guarantee
about the response to inputs in a given state.
GIVEN / WHEN / THEN. methods:behaviours is
MANY-TO-MANY.
read at three levels: the NAME, the given/when/then,
then the code.
common violation: assertions interleaved with calls,
so "when" and "then" blur.
NAME IT AFTER THE BEHAVIOUR . often the ONLY token in
a failure report. verbose is fine -- nothing calls
it and humans read it.
-> needing "AND" in the name = it is two tests.
NO LOGIC IN TESTS . trivially correct on inspection.
one string concatenation hid a doubled separator.
if you need a test for your test, something is wrong.
CLEAR FAILURE MESSAGES . expected, actual, and the
relevant parameters. "account is closed" -- cause,
or unmet expectation? unanswerable.

----- DESCRIPTIVE OVER DEDUPLICATED

deduplication buys easier change -- but tests are
MEANT to be stable, so the benefit is small; and
complexity costs MORE, because tests have no suite
of their own to keep them honest.
the three sharing patterns and their failures:
VALUES -> shared constants hide WHY. use a helper
with defaults, and set only what you care about.
SETUP -> fine for constructing collaborators. if a
test CARES about a value, STATE IT, overriding
the default.
HELPERS -> a catch-all validate() at the end of every

test kills intent and makes one bug fail many.
 good ones assert ONE conceptual fact.
 -> beyond one suite it is TEST INFRASTRUCTURE: many
 callers, hard to change, its own product --
 AND IT MUST HAVE ITS OWN TESTS.

Q Name the four kinds of change and say which should touch existing tests, and why it is dearer.

Q Give both ways an interaction test fails, and the general rule it breaks.

Q Why is unclarity worse in a test than in production code?

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
Testing through the public surface	Tests that break only when a real user would break, plus documentation and examples for users.	More upfront effort than testing the code you just wrote, and you must work out how that code affects the system.	Almost always. It is the single most important thing you can do to keep tests failing only on meaningful change.
Asserting on state rather than the call sequence	A check on what you actually care about, which survives refactoring and catches later corruption.	You need a way to observe the resulting state — which sometimes means a real dependency or a fake rather than a mock.	Whenever the result is observable. Interaction testing is the fallback, not the default.
One test per behaviour rather than per method	Tests that read as sentences, express cause and effect, and show at a glance what is already covered.	More boilerplate and more test methods than mirroring the production structure.	Always for non-trivial methods — the boilerplate is explicitly judged worth it.
Writing the expected value out in full	A test that is trivially correct on inspection, where a wrong expectation is visible rather than computed.	Duplication, and a value you must update by hand if the requirement changes.	Whenever the alternative is logic in the test. Straight-line code beats cleverness here.
Sharing setup across a suite	Tests freed from tedious, irrelevant construction, and shorter bodies.	Completeness, if a test comes to depend on a value defined somewhere the reader cannot see.	For constructing the object under test and its collaborators — and override the default explicitly in any test that cares.
Building shared test infrastructure	Code that makes tests across many suites easier to write, and a standard everyone can learn once.	It behaves like production code: many callers, hard to change safely, and it must carry its own tests.	When the need spans suites. Standardise early on common libraries rather than accumulating several.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO UNDERSTAND IT	SAY IT THIS WAY TO SOUND LIKE AN EXPERT
“My refactor broke thirty tests, so I fixed them.”	“Thirty tests contracted our internals. Either that wasn't a pure refactoring, or they're at the wrong level of abstraction.”
“I made the method public so I could test it.”	“Then I'm testing something no user depends on. What's the behaviour, and how would a caller reach it?”

“The test checks that we call <code>save()</code> .”	“That passes even if the record is deleted a line later. Assert the state instead — read it back.”
“One test per method keeps it tidy.”	“Methods do several things. One test per behaviour, given/when/then.”
“It's called <code>testUpdateBalance</code> .”	“That names the method, not the behaviour. What should happen, in what state? That's the name.”
“I named it <code>shouldRejectAndLogInvalidInput</code> .”	“The 'and' gives it away — that's two behaviours and two tests.”
“I looped over the cases to save repetition.”	“Nothing tests the test. Straight-line code, and I'll take the duplication.”
“The failure just says assertion failed.”	“Then the message needs the expected value, the actual value and the parameters — that's the whole diagnosis.”
“I factored the assertions into a <code>validate()</code> helper.”	“Now no test states its own intent, and one bug will fail all of them. Assert one conceptual fact instead.”
“Where did the string 'Donald Knuth' come from?”	“Setup. If the test cares about the value, it should state it and override the default.”

PART 2 · QUESTION 1

An inventory service has a class that reserves stock. Its tests were written by opening up two private methods, `computeReservationKey` and `writeLedgerRow`, and asserting on the internal map the class uses to stage rows before writing them. A third test verifies, using a mocking framework, that `ledger.append()` is called exactly once with a given argument. Last week an engineer replaced that staging map with a sorted array, inlined one of the two private methods into its only caller, and changed an internal reservation handle from a string to a numeric identifier — behaviour visible to no caller. Nineteen tests failed. In the same week a genuine bug shipped: `append()` only stages a row in an in-memory buffer, which a later `commit()` makes durable, and one error path returns before committing, so reservations taking that path were never written at all. No test noticed. The team's proposal is a rule that private methods must not be renamed without checking the tests first.

(a) Name the category of failure the nineteen breakages belong to, distinguish it from its look-alike, and say which of the four kinds of change this was and what the breakages therefore prove. **(b)** State the practice that would have prevented them, give the argument for why it works, and say what “public” means here. **(c)** Explain precisely why the mocking-framework test did not catch the shipped bug, name both directions in which such a test fails, and give the alternative. **(d)** Assess the team's proposed rule.

.....

.....

.....

.....

.....

.....

PART 2 · QUESTION 2

A suite contains one test per production method. `testApplyDiscount` is 60 lines long and now asserts five different outcomes; it was extended three times as the method grew. Every test calls `checkCartConsistent(cart)` as its last line, a helper that runs eleven assertions. Setup constructs a cart containing an item priced at 1299, and a test named `testApplyDiscount` asserts a total of 1104 with no indication of where either number came from. One test builds its expected receipt line by concatenating a currency symbol, a formatted amount and a suffix, then asserts equality against the same expression rebuilt from the same parts. When the suite fails, the report reads `expected <true> but was <false>`.

(a) Name the structural mistake behind the 60-line test, define the alternative unit, and give three reasons the alternative reads better. (b) Take the shared helper and the setup values together: name what each damages, and give the corrective rule for each. (c) The receipt test asserts against an expression built the same way as the code under test. Say what has gone wrong, name the rule it breaks, and state what should be tolerated instead. (d) Rewrite the failure message so it is diagnosable, and name the three things a good message carries.

2x2 — what the test contracts, and what it asserts

COLUMNS: DOES IT ASSERT ON RESULTING STATE? →

	Yes — it checks what the system became	No — it checks which calls were made
Through the PUBLIC surface	An explicit contract The test uses the system as a user does and checks the outcome a user would notice, so it fails only when a user would be affected. <i>Move:</i> this is the target. Keep it complete and concise, one behaviour per test, named after the behaviour — and let refactoring proceed without touching it.	Right door, wrong question The entry point is fine and the assertion is about the route. It passes when the result is later undone and fails when an equivalent path is substituted. <i>Move:</i> assert the state where you can observe it. Keep the call check only where the result genuinely is the call — Part 3 sets the conditions.
Through INTERNALS	Correct answer, accidental contract The assertion is about the outcome, but reaching it required opening up the implementation — so any tidy-up breaks the test. <i>Move:</i> find the public path that exercises the same behaviour. If none exists, ask whether this is a unit at all or a helper that should be tested through its callers.	Brittleness, doubled Contracted on internal structure and on the sequence of calls. This is the suite that makes a team stop refactoring. <i>Move:</i> treat every failure here as evidence rather than work. Nothing you promised has changed, so the test is asserting something nobody agreed to.

Read it as: the row is *where you knock* and the column is *what you look at*, and the two are independent — which is why fixing only one of them still leaves a brittle test. Both off-diagonal cells are the common half-measures: a team that has learned to use the front door but still asserts call sequences, and a team that asserts outcomes but reaches them through the back.

CARRY-OUT RULE FROM THIS PART

Write the test so that only a change in what the system promises can break it — knock at the front door, assert the outcome rather than the route, and keep it readable enough to be checked by eye, because nothing tests the test.

Standing in for the real thing

The three ways to substitute a dependency, what each one costs in truthfulness, and why the real implementation usually wins.

TENTH-GRADER VERSION

- Some things can't be in a test — a real payment service, a real database. So you put in a stand-in.
- Three kinds. A *fake* is a simple working version. A *stub* just returns whatever answer you told it to. An *interaction check* doesn't answer at all — it only records that it was called.
- First choice is still the real thing. Tests that run the actual code tell you more.
- If the real thing is too slow, too unpredictable, or too painful to build, then substitute.
- A fake is best, because the code being tested can't tell the difference. But somebody has to write it and keep it honest — so it needs its own tests.
- Stubbing is quick and it hardcodes an assumption. If the assumption is wrong, your test passes and the system is broken.
- Checking which calls were made is the weakest option: it can only tell you the code *tried*, never that it worked.

THE EVERYDAY VERSION

A flight school cannot practise engine failure by switching off a real engine over a city. So it substitutes. A **simulator** is a working model: it flies wrong when you fly it wrong, so the trainee cannot tell the difference from inside — which is exactly what makes it useful, and exactly why somebody has to keep it matched to the real aircraft as the aircraft changes. A **scripted answer** is cheaper: the instructor simply says “assume the left engine is out”. Fast, flexible, able to produce situations no simulator would generate — and it teaches whatever the instructor happened to assume, so if the real aircraft yaws the other way, the trainee has practised the wrong thing. And the weakest substitute is a **checklist observer** who does not fly anything and only records that the trainee reached for the correct lever. That confirms the drill was attempted. It cannot confirm the aircraft would have stayed in the air. All three have their place, and the ranking never changes: fly the real aircraft where you safely can, simulate where you cannot, script sparingly, and treat “they reached for the lever” as the last resort it is.

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
A test double	An object or function that can stand in for a real implementation in a test , as a stunt double stands in for an actor.	The need is created by Part 1's constraints: production code often does not fit inside the limits of a small test, because it communicates across processes or machines. Test doubles can be much more lightweight, letting you write many small tests that execute quickly and are not flaky. They also make it easy to trigger a rare error condition, or to ensure a heavyweight function is called without executing it.	The word mocking is often used for all of this, and is avoided here because it also names something narrower. Substitution introduces three complications. Testability — the codebase must be designed so tests can swap implementations, and retrofitting that later can require a major refactor. Applicability — improper use produces tests that are brittle, complex and less effective, and the damage is magnified across a large codebase. Fidelity — how closely the double's behaviour resembles the real thing.
Seams and dependency injection	Code is testable if it is written so unit tests can be written for it. A seam is what makes that possible: a way to use different dependencies for the system under test than the ones used in production.	Dependency injection is the common technique: rather than a class instantiating what it needs, those dependencies are passed to it , so they can be substituted in tests. Production passes the implementation that talks to an external server; the test passes a double. Frameworks exist to construct object graphs automatically and remove the boilerplate.	vs a universal requirement. In dynamically typed languages it is possible to replace individual functions or methods directly, which makes injection less important there, because you can keep the real dependency and override only the parts unsuitable for tests. The timing point is the one to keep: writing testable code requires an upfront investment, and it is especially critical early, because the later testability is considered the harder it is to apply — code written without testing in mind typically needs refactoring or rewriting first.
Technique 1 — faking	A fake is a lightweight implementation of an interface that behaves similarly to the real implementation but is not suitable for production — an in-memory database, an in-memory file system.	It is often the ideal technique when you need a double , because it behaves like the real thing: the system under test should not be able to tell whether it is interacting with a real implementation or a fake. It stores state, so a value written can be read back.	vs a convenience. A fake is real work: one might not exist for the object you need, and writing one is challenging because you must ensure it has similar behaviour to the real implementation now and in the future. That maintenance burden is why ownership matters, treated three rows below.
Technique 2 — stubbing	Giving behaviour to a function that otherwise has no behaviour of its own — you specify exactly what values it returns.	Typically done through a mocking framework, to avoid the boilerplate of hand-writing classes that hardcode return values. It is quick and simple to apply.	vs faking. A fake <i>implements</i> the contract; a stub <i>asserts</i> a fragment of it. That difference generates all three of stubbing's dangers, two rows below.
Technique 3 — interaction testing	Validating how a function is called without actually calling the implementation of the function. The test fails if it is not called, is called too many times, or is called with the wrong arguments.	Also typically done through a mocking framework, which removes the boilerplate of hand-writing classes that count calls and record arguments.	This is the thing that is sometimes called mocking — a terminology avoided here because mocking frameworks do stubbing as well. It is useful in certain situations but should be avoided when possible, because overuse easily results in brittle tests. Conditions are set out in the last two rows.

The real implementation first	The first choice is the same implementations used in production code. Tests have higher fidelity when they execute code as it will be executed in production.	Preferring real implementations is called classical testing; preferring frameworks is called mockist. Realism beats isolation because a test full of doubles isolates the system from its dependencies, and then an engineer may need integration tests or manual checks to reach the same confidence — extra work that slows development and lets bugs through when people skip it.	Two arguments beyond fidelity. Replacing all dependencies arbitrarily isolates the system to whatever the author happened to put in that class and excludes implementation that happens to live in another class — but a good test should be written in terms of the API being tested rather than how the implementation is structured. And when a real dependency has a bug and your test fails, this is good: it means your code will not work properly in production, and good continuous-integration tooling makes it easy to find the change responsible.
When a real implementation is not the answer	A real implementation is preferred if it is fast, deterministic and has simple dependencies — a value object, a date, an amount of money, a collection. Three considerations decide the rest.	Execution time: there is no exact threshold. One extra millisecond per case is nothing; one extra second may be reasonable across five cases and not across five hundred. For borderline cases it is often simpler to use the real implementation until it becomes too slow, then switch. Determinism: a real implementation is more complex and so likelier to be nondeterministic — multithreading whose output depends on scheduling, code that is not hermetic and depends on an external service, code that reads the system clock. Dependency construction: a real object needs its entire dependency tree built, and those tests need maintenance whenever a constructor signature changes.	vs a rule you can apply mechanically — the material is explicit that trade-offs must be made. Two counterweights. Rare flakiness might not warrant a response, because there is little disruption; frequent flakiness means it is time to substitute. And on construction, the ideal answer is not a double but the same object-construction code used in production — a factory method or automated injection — which requires that code to be flexible enough to accept doubles.
The fidelity of a fake	How closely the behaviour of a fake matches the real implementation. If they diverge, a test using the fake is not useful — it might pass while the same code path fails against the real implementation.	Perfect fidelity is not always feasible; the fake exists precisely because the real thing was unsuitable, so a fake database will not match a real one on disk storage. Primarily, a fake should maintain fidelity to the API contracts of the real implementation: for any given input it should return the same output and make the same state changes — if saving an item succeeds for a new identifier and errors for an existing one, the fake must do the same.	One way to think about it: the fake must have perfect fidelity to the real implementation, but only from the perspective of the test. A fake hashing function need not produce the same hash values, only unique ones per input, provided the contract makes no promise about specific values. Latency and resource consumption are usually outside the needed fidelity too — but you cannot use a fake if you need to test those constraints explicitly. And a fake need not implement everything: for unsupported paths it is best to fail fast, which tells the engineer the fake is not appropriate here.

Who owns a fake, and why it needs tests	Because a fake must track the real implementation's behaviour as that behaviour changes, the team that owns the real implementation should write and maintain the fake.	The economics are explicit: a fake requires more effort and more domain expertise to create, and a team should weigh whether the productivity gain outweighs the cost — for a handful of users it may not be worth it; for hundreds it is an obvious win. To limit how many exist, create a fake only at the root of the code that is not feasible to use in tests: if a database cannot be used, fake the database interface itself rather than every class that calls it.	The correctness requirement is absolute: a fake must have its own tests to ensure it conforms to the API of its real counterpart , or its behaviour will diverge over time as the real implementation evolves. The technique named for this is a set of tests written against the public interface and run against both the real implementation and the fake — the real-implementation runs are slower, but only the fake's owners need to run them. If no fake exists: ask the owners, since they may not know the concept or its benefit; failing that write your own by wrapping the calls in a single class and faking that; failing that, fall back to the real implementation or another technique.
The three dangers of overusing stubbing	Because stubbing is so easy to apply, it is tempting whenever a real implementation is not trivial. Overuse costs three specific things.	Tests become unclear: the extra code defining stubbed behaviour detracts from the intent of the test and is hard to follow if you do not know the implementation. Tests become brittle: stubbing leaks implementation details of your code into your test , so a change to how the code works forces a test update, when a good test should change only if user-facing behaviour changes. Tests become less effective: there is no way to ensure the stubbed function behaves like the real one , so you duplicate part of the contract with no guarantee it is correct — and there is no way to store state , so you cannot save an item and then read it back.	The diagnostic worth memorising: a key sign that stubbing is not appropriate is finding yourself mentally stepping through the system under test in order to understand why certain functions are stubbed. The worked contrast is stark — a test with five stubbed calls can only end by verifying that one of them happened, whereas removing the stubbing lets the same test query the resulting state, and it becomes shorter, needs no special setup, and stops exposing how the internals fit together.
When stubbing is appropriate	Not as a catch-all replacement, but when you need a function to return a specific value to get the system under test into a certain state.	Because the behaviour is defined inline, stubbing can simulate a wide variety of return values or errors that might not be possible to trigger from a real implementation or a fake — which is its unique capability.	Two constraints keep it honest. Each stubbed function should have a direct relationship with the test's assertions , so its purpose is clear. And a test should stub a small number of functions, because a test requiring many stubs is a sign that stubbing is being overused, or that the system under test is too complex and should be refactored. Even where appropriate, real implementations and fakes remain preferred: they do not expose implementation details and they give more guarantees about correctness.

When interaction testing is warranted	State testing is preferred; interaction testing is the fallback. Its primary issue: it cannot tell you that the system under test is working properly — only that certain functions were called as expected.	That forces an assumption — if the save call is made, we assume the item was saved — whereas state testing actually validates the assumption , by saving and then querying. It also uses implementation details, since proving a function was called exposes to the test that the system calls it. Tests that overuse it are jokingly called change-detector tests , because they fail in response to any change to production code even when the behaviour is unchanged . Two cases warrant it: when you cannot perform state testing because no real implementation or fake is usable; and when differences in the number or order of calls would cause undesired behaviour — verifying that a cache reduces database reads, for example.	Two practices reduce the damage. Perform interaction testing only for state-changing functions — those with side effects on the world outside the system under test, such as sending an email or saving a record — because for non-state-changing functions the check is usually redundant : the system will use the return value to do work you can assert on, and the interaction itself is not important for correctness. And avoid overspecification : verify the minimum necessary for the behaviour under test, so unrelated changes do not fail the test and a reader can tell which assertions matter. Finally, it is not a complete replacement — where a unit test can only check interactions, consider supplementing with a larger-scoped test that checks state.
--	---	--	---

CONCEPT BOUNDARY — EVERY SUBSTITUTION IS A PURCHASE, AND FIDELITY IS THE CURRENCY

What it sounds like: a menu of interchangeable tools for the same job. You need the database out of the test, so pick whichever technique is most convenient — the framework makes all three about equally easy to write.

What it actually is: a ranked ladder, and each rung down buys speed and control by spending truthfulness. The real implementation runs the code that will run in production. A fake runs different code that honours the same contract, so the system under test cannot tell — which is why it is the best substitute and also why somebody must keep it matched. A stub honours no contract at all; it replays one answer the test author believed was right, so it encodes an assumption that nothing checks. And an interaction check does not even claim an answer: it records that a call happened, which is compatible with the operation having done nothing, or having been undone a line later.

Why that matters practically: it tells you what a green test is worth. With a real implementation or a fake, green means the behaviour happened. With a stub, green means the behaviour happened *if* the author's belief about the dependency was accurate. With an interaction check, green means the attempt was made. Those are three different claims, and the last one is compatible with a broken system — which is why the fallback carries an instruction to supplement it with a larger-scoped test that checks state.

The trap: letting ease of writing choose. The framework makes the weakest options the fastest to type, and the cost arrives later and elsewhere — in tests that need constant maintenance while rarely finding bugs, and in an interface owner discovering that thousands of tests have hardcoded beliefs about their contract, some of which were never true.

Anchor sketch — the ladder of substitutes, and what each one costs

Legend: bold = a named technique or rule
this

reversed = the fact everything else answers

Q = cover the page and answer

PRODUCTION CODE OFTEN WILL NOT FIT INSIDE A SMALL TEST
-- it crosses processes or machines. so we substitute.

THREE COMPLICATIONS COME WITH THAT
TESTABILITY the code must ALLOW swapping. retrofit
it late and you are refactoring.
APPLICABILITY misuse => brittle, complex, less
effective -- magnified across a codebase.
FIDELITY how close the stand-in is to the real thing.

|
SEAMS . a way to use DIFFERENT dependencies in the test.
DEPENDENCY INJECTION: pass them in, do not construct
them inside. (less needed in dynamic languages, where
you can override just the unsuitable methods.)
-> testability is an UPFRONT investment. late is dear.

**THE LADDER: EACH RUNG DOWN BUYS SPEED AND CONTROL
BY SPENDING TRUTHFULNESS.**

0 THE REAL IMPLEMENTATION -- FIRST CHOICE.

highest fidelity: it runs the code production runs.
replacing every dependency ARBITRARILY isolates you
to whatever the author put in THIS class.
if a real dependency's bug fails your test, GOOD --
your code would not work in production either.
prefer it when FAST + DETERMINISTIC + SIMPLE DEPS
(value objects, dates, money, collections).
when NOT: time (no fixed threshold; 1ms x 5 is fine,
1s x 500 is not -- use it until it is too slow) .
determinism (threads, the clock, non-hermetic calls;
RARE flakiness may not warrant a response) .
construction (the whole dependency TREE -- but the
right fix is production's own factory or injection).

1 FAKE -- best substitute. a lightweight working
implementation. THE SYSTEM UNDER TEST CANNOT TELL.
it holds STATE: write, then read back.
FIDELITY = to the API CONTRACT, and only FROM THE
PERSPECTIVE OF THE TEST. a fake hash need not match
the real hash, only be unique per input. but you
CANNOT fake what you are explicitly testing (latency).
unsupported paths: FAIL FAST.
OWNED BY THE TEAM THAT OWNS THE REAL THING, because
it must track it. build it at the ROOT of what is
untestable -- the database API, not every caller.
A FAKE MUST HAVE ITS OWN TESTS. run one set of
tests against BOTH the real thing and the fake.
Q no fake exists -- what are the four moves, in order?

2 STUB -- hardcode a return value. quick, and:

UNCLEAR the setup code drowns the intent.
BRITTLE it LEAKS IMPLEMENTATION into the test.
WEAK nothing guarantees the stubbed answer
matches the real contract -- you duplicate
the contract with no way to check it. and
NO STATE: you cannot save then read back.
-> THE TELL: you are mentally stepping through the
system under test to work out WHY something is
stubbed.
-> when it IS right: to force the system into a
particular state, and to produce returns or
errors a real thing or a fake CANNOT trigger.

each stub must relate DIRECTLY to an assertion.
MANY stubs = overuse, or the code needs refactoring.

3 INTERACTION TEST -- last resort. it validates the CALL, not the result. so it can only say the code TRIED. it assumes what state testing would have PROVEN. overused, these are CHANGE-DETECTOR TESTS: they fail on ANY production change, behaviour unchanged. warranted: (a) state testing is impossible -- no real implementation, no fake. (b) the NUMBER or ORDER of calls itself matters (a cache must not re-read). do it only for STATE-CHANGING functions (send, save, log) -- for non-state-changing ones (get, find, read) it is REDUNDANT, since you can assert on what the system did with the return value. AVOID OVERSPECIFICATION: verify the minimum. -> not a replacement. supplement with a larger test that checks STATE.

Q What does a green test mean at each of the four rungs?

Q Give the three dangers of overused stubbing, and the tell that you have overused it.

Q Which functions may be interaction-tested, and why are the others redundant?

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
Keeping the real dependency	The highest fidelity available — the code that runs in production runs in the test, and its bugs surface as your failures.	Time, potential nondeterminism, and the whole dependency tree to construct.	Whenever it is fast, deterministic and simply constructed. For borderline cases, keep it until it becomes too slow.
Writing a fake	A substitute the system under test cannot distinguish from the real thing, holding state and honouring the contract.	Real effort and domain expertise to write, ongoing maintenance as the real implementation evolves, and its own tests.	When the real implementation cannot be used and there are enough users to repay the cost. Build it at the root of what is untestable.
Stubbing a return value	Speed, and the ability to produce returns or errors that no real implementation or fake could be made to trigger.	Clarity, resilience to implementation change, any guarantee of contract fidelity, and the ability to store state.	To force the system into a particular state, with each stub tied directly to an assertion — and only a few per test.
Verifying that a call was made	Some basic confidence when nothing else is available, and the only practical check on call count or ordering.	It cannot show the system works, it hardcodes an assumption, and it exposes internals — the recipe for a change-detector test.	When state testing is impossible, or when the number or order of calls is itself the behaviour. State-changing functions only.

Building seams into the code	The ability to substitute anything later, and code that tends to be more modular.	Upfront design work, and boilerplate unless an injection framework absorbs it.	Early. The later testability is considered, the more likely the code needs rewriting to accept it.
Supplementing a doubled unit test with a larger one	A check on the assumption the double encoded — real state, verified against the real thing.	A slower test with all of Part 4's costs attached.	Whenever a unit test could only validate interactions. This is explicitly not optional risk management.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO <i>UNDERSTAND IT</i>	SAY IT THIS WAY TO SOUND LIKE AN <i>EXPERT</i>
“I mocked the dependency.”	“Which technique — a fake, a stub, or a call check? They give three different guarantees.”
“Constructing the real one is a pain, so I used a double.”	“Then the fix is production's own factory or injection, not a double. Construction cost isn't a fidelity argument.”
“The real dependency has a bug and my test fails.”	“That's the test working. My code won't work in production either.”
“Our fake returns whatever is convenient.”	“Then it has no fidelity to the contract. Same inputs, same outputs, same state changes — and it needs its own tests.”
“I'll write a fake for each class that calls the database.”	“Fake the database interface itself. Build it at the root of what's untestable, or you'll maintain a dozen.”
“There are eight when-then lines before the assertion.”	“If I'm mentally stepping through the code to see why each one is there, stubbing is the wrong tool here.”
“The stub returns 3 for add(1,2), which is right.”	“I've duplicated part of the contract with no way to check it's still true. Nothing keeps that honest.”
“I verified that save() was called.”	“That says we tried. Read it back instead — otherwise a deletion two lines later still passes.”
“Every one of our tests broke when I renamed that call.”	“Change-detector tests. They're asserting the route, and the behaviour never changed.”
“I verified getUser() was called too.”	“That one doesn't change anything, so the check is redundant — assert on what we did with the value.”

PART 3 · QUESTION 1

A notifications service decides which subscribers to email and records each send. Its tests construct the system with framework-created doubles for the subscriber store, the template renderer, the delivery client and the audit log. A typical test declares six canned return values, calls `notifyAll()`, and finishes by verifying that `deliveryClient.send()` was called and that `subscriberStore.findActive()` was called. The team reports that the tests are fast and that they break constantly — a recent change that split `findActive()` into two calls failed 140 tests with no behaviour change. Separately, production has been silently dropping sends for a fortnight: the delivery client returns a receipt object whose `accepted` flag the service never checks, and every canned return value in the suite sets that flag to true. The subscriber store's owning team publishes no fake; nobody has asked them for one.

(a) Name the two techniques in use and say, for each, exactly what a green test here is claiming. **(b)** Explain the 140 failures using the correct term, and name the general property being asserted instead of the intended one. **(c)** Explain how the suite stayed green for a fortnight, naming the specific danger and the guarantee stubbing cannot give. **(d)** Give the four moves available when no fake exists, in order, and say who should own the fake and where in the code it belongs.

PART 3 · QUESTION 2

A team is writing tests for a booking engine. Four dependencies are in play. A money type with no dependencies, which the team has replaced with a double because “doubles are the standard here”. A pricing library that is real, deterministic, and adds 1.2 milliseconds per case across 900 cases; the team wants to double it. A seat-allocation component that reads the system clock and occasionally produces a different result for the same input, causing about one failure a week. And a payments gateway with an official in-memory fake, which the team has not adopted because their tests need to exercise the rare “issuer declined after authorisation” path, which the fake cannot produce. The team also wants to prove that a new de-duplicating wrapper collapses a burst of identical booking-confirmation requests into exactly one call to the external delivery provider.

(a) Rule on the money type and the pricing library, giving the criterion for each and the arithmetic where it applies. **(b)** Rule on the seat-allocation component: name the two causes of its behaviour, and give the stated test for whether flakiness warrants a response. **(c)** The team's objection to the fake is legitimate. Name the technique that answers it, state the two constraints that keep it honest, and say what the fake should do about the path it does not support. **(d)** Rule on the cache requirement: name the technique, justify it against the general preference, and give the restriction on which functions it may be applied to.

2x2 — does the stand-in honour the contract, and can you see the result

COLUMNS: CAN THE TEST OBSERVE THE RESULTING STATE? →

	Yes — write it, then read it back	No — all you can see is the call
HONOURS the contract (real, or a tested fake)	Green means it happened The code that runs is either production's own or something the system under test cannot distinguish from it, and the assertion is on what the system became. <i>Move:</i> this is the target rung. Keep the fake owned by the interface's owners, built at the root of what is untestable, and covered by tests run against both it and the real thing.	Faithful, and only watching the door A trustworthy stand-in whose effect you cannot inspect — so the check falls back to the call. Legitimate where the call count or ordering is itself the behaviour. <i>Move:</i> restrict it to state-changing functions, verify the minimum, and add a larger-scoped test that checks the state you could not see here.
HONOURS NOTHING (stub, or a bare double)	Green means it happened, if you guessed right You can read something back, but what you read is what you told it to say. The test now encodes an assumption about the dependency that nothing verifies. <i>Move:</i> acceptable to force a state you cannot otherwise reach. Tie each canned value to an assertion, keep them few, and do not let them stand in for the contract.	Green means it tried Nothing honours the contract and nothing observes an outcome. Compatible with a system that does the wrong thing, or undoes it a line later. <i>Move:</i> the cell where change-detector tests live. Treat it as a fallback with a debt attached — and the debt is a larger test that checks real state.

Read it as: the row is *fidelity* and the column is *observability*, and together they determine what a passing test entitles you to say. Reading clockwise from the top left, the claim degrades from “the behaviour occurred” to “the behaviour occurred if my assumption held” to “an attempt was made” — and the bottom-right cell is the one people arrive at by accident, because it is the fastest to write.

CARRY-OUT RULE FROM THIS PART

Use the real thing while it is fast, deterministic and simple to build; substitute the most faithful stand-in you can afford; and know what a green test entitles you to claim at each rung down.

What small tests cannot tell you

The gaps a vacuum leaves, the quantity that closes them, and what a larger test costs to build, run and own.

TENTH-GRADER VERSION

- Small tests are fast and reliable because they cut the system off from the real world. That's also what they can't see.
- Five blind spots: stand-ins that don't match reality, broken configuration, behaviour under heavy traffic, things nobody thought of, and behaviour that only appears when the parts run together.
- Bigger tests fix those and break every rule that made the small ones nice: they're slow, they're flaky, and they're hard to run lots of.
- Two more problems that are about people, not machines: nobody's sure who owns them, and everyone builds them differently.
- Every big test has the same four steps: get a copy of the system, put data in it, do something, check what happened.
- The copy can be more realistic or more isolated. You usually can't have both.
- Keep them as small as you can while still being realistic — and make sure a failure tells you something you can act on.

THE EVERYDAY VERSION

A car maker tests a new brake design on a rig: the caliper is clamped in a jig, a motor spins a disc, and a sensor measures the stopping torque a thousand times an hour, every result identical. It is fast, repeatable, and it will catch a bad batch of pads immediately. But notice what the jig has removed in order to be repeatable — the weight of the car, the temperature of a mountain descent, the wiring loom, the software that decides how hard to squeeze, the wheel it was bolted to by a person, and the rain. Every one of those is a real cause of brake failure and **not one of them can appear on the rig**, because removing them is exactly what made the rig fast. So you also drive the car. That costs a track, a day, a driver, and it produces results that vary between runs, which is precisely the price of putting the world back in. The rig is not wrong; it simply answers a smaller question than the one you eventually have to answer. And the crucial detail: **if the jig's sensor and the real wheel disagree by ten percent, every one of those thousand identical results is ten percent wrong, all at once.**

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
Larger tests	Tests not bound by the small-test constraints. They may be slow, may be nonhermetic, and may be nondeterministic .	Default timeouts of fifteen minutes or an hour are normal, and some run for hours or days. They may share resources with other tests and with traffic — and once a test is nonhermetic, it is almost impossible to guarantee determinism , because other tests or user state may interfere.	vs a failure of discipline. The justification is a question about your own habits: you run unit tests as you go, and then you run the actual binary and try it out — and when you hand the code to somebody else, they try it out too. Unit tests give confidence about individual functions, objects and modules; large tests give more confidence that the overall system works as intended — and automating that scales in a way manual trying-out does not.
Fidelity	The property by which a test is reflective of the real behaviour of the system under test. It is the primary reason larger tests exist.	One way to see it is the environment: a unit test bundles a test and a small portion of code into a runnable unit, which ensures the code is tested but is very different from how production code runs ; production itself is the environment of highest fidelity, with a spectrum of interim options between. The other way is the test <i>content</i> : hand-crafted data covers a narrow range of cases and tends to conform to the biases of the author , while data copied from production is more faithful precisely because it was captured that way.	vs realism as an unqualified good. Increasing fidelity also comes with increasing costs and — in the case of production — increasing risk of failure , so the job is to find the proper fit rather than to maximise it. The scenarios that hand-crafted data misses are named directly: they are a fidelity gap in the tests .
Gap 1 — unfaithful doubles	When a heavyweight dependency is replaced, it becomes possible that the replacement and the doubled thing do not agree .	The mechanism is social. Almost all unit tests are written by the same engineer writing the unit under test, so when doubles are needed it is that engineer who defines the double and its intended behaviour — and they usually did not write the thing being doubled and can be misinformed about its actual behaviour . The relationship between a unit and its peer is a behavioural contract; if the engineer is mistaken about the actual behaviour, the understanding of the contract is invalid .	And doubles become stale : if the double is not visible to the author of the real implementation and that implementation changes, there is no signal that the test and the code should be updated. Note the escape hatch, which is Part 3's conclusion restated: if teams provide fakes for their own services, this concern is mostly alleviated .

Gap 2 — configuration	Unit tests cover code within a binary, but that binary is typically not completely self-sufficient in terms of how it is executed.	There is usually a deployment configuration or a starter script, and real production instances have their own configuration files or databases. Problems in those files, or incompatibility between the state they define and the binary, can lead to major user issues that unit tests alone cannot verify.	The scale of this gap is the striking part: configuration changes are the number one reason for major outages. Configurations tend to be written in configuration languages rather than production languages, they often have faster rollout cycles than binaries, and they can be harder to test — all of which raises the likelihood of failure. Hence the practice attached: keep configuration in version control alongside code , so that a change to it can be identified as the source of a bug rather than appearing as random external flakiness — and so it can be built into larger tests.
Gap 3 — behaviour under load	Performance, load and stress testing often require sending large volumes of traffic to a binary , which does not fit the model of a typical unit test.	Unit tests are meant to be small and fast so they fit standard execution infrastructure and can run many times in a frictionless workflow. The volumes needed to provoke load problems can be thousands or millions of queries per second.	Load and stress handling are described as highly emergent properties: complex behaviours that belong to the overall system but not to its individual members — which is why they cannot be decomposed into smaller tests, and why such tests must be made to look as close to production as possible.
Gaps 4 and 5 — the unanticipated, and the vacuum	Unit tests are limited by the imagination of the engineer writing them — they can only test for anticipated behaviours and inputs.	But the issues users find are mostly unanticipated ; if they had been anticipated they would be unlikely to reach users at all. Hyrum's Law sharpens this: even if you could test completely for conformance to a strict specified contract, the effective user contract applies to all visible behaviours, not just a stated one , and unit tests alone are unlikely to cover every visible behaviour outside the declared interface.	The fifth gap is the summary of all of them. Unit tests are limited to the scope they cover, and because they are designed to be fast and reliable they deliberately eliminate the chaos of real dependencies, network and data. The image to keep: a unit test is like a problem in theoretical physics — ensconced in a vacuum, neatly hidden from the mess of the real world , which is great for speed and reliability and misses certain categories of defect.
Why not have larger tests	A developer-friendly test must be reliable (not flaky, with a useful pass/fail signal), fast (not interrupting the workflow), and scalable (runnable efficiently across the whole change stream). Larger tests often violate all three.	They are flakier because they use more infrastructure; slower both to set up and to run; and hard to scale because of resource and time requirements and because they are not isolated — they can collide with one another.	Two further challenges are organisational rather than technical. Ownership: a unit test is clearly owned by whoever owns the unit, but a larger test spans multiple units and therefore multiple owners — and without clear ownership a test rots. Standardisation: unlike unit tests, larger tests lack standard infrastructure and process, because the approach is a product of a system's architectural decisions. The consequences are concrete: they are often skipped during large-scale changes because there is no standard way to run them, testing the integration between two teams requires unifying incompatible infrastructures, and no single approach can be taught to newcomers — which perpetuates the situation.

Why fidelity compounds	The argument for taking fidelity seriously at scale, and against papering over it with doubles.	Picture the system as a graph of services: every new node has a multiplicative effect on the number of distinct execution paths, so the number of scenarios to test end-to-end grows exponentially or combinatorially and does not scale. But the value of such tests rises too, because of the same structure: if service doubles are of lower fidelity, the chance of bugs when putting it all together is exponential in the number of them. With just two doubles that are each 10% inaccurate, the likelihood of a bug is 99%.	The practical instruction that follows is counterintuitive for a chapter about big tests: even for integration tests, smaller is better — a handful of large tests is preferable to an enormous one. Because a test's scope is coupled to the scope of the system under test, making that system smaller is what makes the test smaller. And where a journey needs contributions from many internal systems, the technique is to chain it: several smaller pairwise tests where the output of one is persisted and used as the input to the next.
The four phases, and the system under test	Most larger tests share a workflow: obtain a system under test; seed the necessary data; perform actions using it; verify the behaviours.	The system under test is judged on two factors. Hermeticity — its isolation from usage and interaction other than the test in question; high hermeticity means the least exposure to concurrency and infrastructure flakiness. Fidelity — its accuracy in reflecting the production system, meaning binaries that resemble the production versions, similar configuration, similar infrastructure, similar topology. The forms run from a single-process system (everything in one binary, possibly with the test inside it — least faithful to production topology, but can be a small test), through single-machine and multi-machine, to a shared environment such as staging or production, with hybrids in between.	The sentence to keep is that these two factors are often in direct conflict. Shared environments cost least because they already exist, and they force the test to wait until code has been pushed there, conflict with other simultaneous users, and — in production — risk end-user impact. A test in production cannot block the release of the code to that environment: the system under test is too late. Reserving windows in a shared staging environment does not scale, as the environment, its users and the likelihood of conflicts all grow.

Shrinking the system at problem boundaries	Since the test's scope follows the system's scope, the highest-leverage move is choosing where to cut.	Two boundaries are singled out. Interfaces: tests that involve both a front end and a back end are painful because such tests are notoriously unreliable and costly — the look and feel changes in ways that break tests without affecting behaviour, and asynchronous behaviour is hard to test. Where the back end offers a public interface it is often easier to split the tests at that boundary and drive the end-to-end tests through it. Third parties: they may offer no shared environment for testing and there can be a cost per call, so it is not recommended to have automated tests use a real third-party interface — that dependency is an important seam at which to split.	The other technique is record and replay. A larger test runs continuously and its primary purpose is to generate traffic logs by recording calls to external services; a smaller test replays them during development. Because of nondeterminism, requests must be matched to decide which response to replay — which makes it very similar to stubbing, in that argument matching determines the resulting behaviour. When client behaviour changes significantly a request may no longer match, so the recording run must be easy, fast and stable to repeat.
Test data and verification	A large test needs two kinds of data: seeded data, preinitialised into the system to reflect its state at the start, and test traffic, sent to it during execution.	Seeding is orders of magnitude more complex than a unit test's setup. Three difficulties: domain data that must be prepopulated or the binaries fail on startup; a realistic baseline, since a system will not be perceived as realistic without enough base data of the right quality and quantity — a social network's tests need a realistic graph before anyone accepts them; and seeding interfaces, since writing directly to a datastore may bypass the triggers and checks the real binaries perform. Data can be hand-crafted, copied from production, or sampled — where smart sampling copies the minimum needed for maximum coverage.	Verification comes in three kinds with different costs. Manual, where humans interact with the system — and manual regression testing does not scale sublinearly: the larger the system and the more journeys through it, the more human time is needed. Assertions, as in a unit test — explicit checks on intended behaviour. And differential comparison, which runs two copies, sends the same data and compares outputs — where the intended behaviour is not explicitly defined, so a human must go through the differences to decide whether the changes were intended.

Keeping larger tests usable	Four disciplines: make them fast, drive out flakiness, make failures intelligible, and give them owners.	Speed: the best move is to reduce scope or split one test into two that run in parallel. Replace time-based sleeps with polling at microsecond frequency, an event handler, or a subscription — tests that rely on sleeps and timeouts all start failing when the fleet becomes overloaded, which spirals, because those tests are then rerun more often. Make production's internal timeouts tunable, since a system configured for a distributed deployment may be running on one machine here. Flakiness: minimising it starts with reducing scope — a hermetic system avoids the multi-user flakiness of a shared environment, and a single-machine one avoids network and deployment flakiness.	There is a subtle trap in internal timeouts: a production system limits end-user exposure by handling internal failures gracefully, so a service that cannot produce a result in time may return a degraded response rather than an error — which looks to a test runner like broken code when it is actually a flaky timeout. Understandability: a stack trace is not useful when the call chain crosses process boundaries, so a good failure must name what failed in context rather than saying “assertion failed”, produce a trace across the call chain or automation that narrows the culprit, and provide support and contact information. Ownership: larger tests must have documented owners who can review changes and support failures; integration tests of one project belong to its lead, and feature-focused tests to a feature owner, who must be empowered, able to support maintenance, and incentivised to do so.
-----------------------------	--	---	---

CONCEPT BOUNDARY — THE VACUUM IS THE FEATURE, AND THE BILL FOR IT COMES HERE

What it sounds like: a hierarchy of thoroughness. Unit tests are the quick check, larger tests are the real check, and a mature team simply has more of the latter — with the pyramid as a budgeting compromise for teams who cannot afford them.

What it actually is: a statement that the two kinds answer different questions because they were built by removing different things. A small test is fast and repeatable *because* it has removed the network, the configuration, the concurrency, the volume and the other services. Those removals are not incidental — they are the entire source of its speed and determinism, and they are simultaneously the exact list of things it can no longer see. That is why the five gaps read like an inventory of what the vacuum excluded, and why no quantity of unit tests closes them.

Why that matters practically: it stops you arguing about coverage and starts you arguing about risk. The question a larger test answers is never “is this code correct” — smaller tests do that better and cheaper. It is “does this hold once the removed things are put back”: does the configuration load, does it survive the volume, do the doubles match what the real service does, does anything emerge that no single component expresses. And that framing tells you which larger test to write, because each type in the catalogue puts back a different removed thing.

The trap: treating fidelity as free, and therefore always choosing more of it. Fidelity conflicts with hermeticity, and every step toward the real environment buys truthfulness with reliability, speed and — at the production end — actual user risk. Hence the instruction that sounds paradoxical in a chapter about big tests: make them as small as you can while still retaining the fidelity you needed them for, because the scope of the test follows the scope of the thing you chose to stand up.

Anchor sketch — what the vacuum removed, and what putting it back costs

Legend: bold = a named gap or property

reversed = the fact everything else answers

Q = cover the page and answer this

A UNIT TEST IS ENSCONCED IN A VACUUM -- HIDDEN FROM THE MESS OF THE REAL WORLD. THAT IS WHAT MAKES IT FAST,

AND IT IS THE LIST OF WHAT IT CANNOT SEE.

|
FIDELITY = how far a test reflects the REAL behaviour of the system. THE PRIMARY REASON LARGER TESTS EXIST.

two readings: the ENVIRONMENT (unit test <-> ... <-> production, the highest fidelity of all) and the CONTENT (hand-crafted data is narrow and carries the AUTHOR'S BIASES; copied production data is faithful because it was captured that way).

-> more fidelity = more cost, and in production, MORE RISK. find the fit; do not maximise.

FIVE GAPS UNIT TESTS LEAVE

UNFAITHFUL DOUBLES . the SAME ENGINEER writes the unit and its doubles -- and did NOT write the thing being doubled. get the contract wrong and the understanding is invalid. worse, doubles GO STALE: the real author never sees them, so nothing signals a change.
-> mostly alleviated IF teams ship fakes for their own services.

CONFIGURATION . the binary is not self-sufficient: deployment config, starter scripts, config databases. CONFIG CHANGES ARE THE NUMBER ONE CAUSE OF MAJOR OUTAGES -- different languages, faster rollouts, harder to test.
-> keep config IN VERSION CONTROL, so a bug traces to a change instead of looking like flakiness.

LOAD . thousands to millions of queries per second. load and stress are HIGHLY EMERGENT: they belong to the system, not to any member -- so they cannot be decomposed.

THE UNANTICIPATED . unit tests are bounded by the IMAGINATION OF THEIR AUTHOR, and real user issues are mostly unanticipated -- or they would not have reached users. HYRUM'S LAW: the effective contract is ALL VISIBLE BEHAVIOUR, not the stated one.

EMERGENT BEHAVIOUR . outside the covered scope, and the chaos was deliberately removed.

WHY NOT JUST WRITE BIG TESTS? they break all three developer-friendly properties: RELIABLE . FAST . SCALABLE (flakier -- more infrastructure; slower to set up AND run; and NOT ISOLATED, so they collide).

plus two human problems:

OWNERSHIP -- a big test spans many owners. WITHOUT CLEAR OWNERSHIP A TEST ROTS.

STANDARDISATION -- no common way to write or run them, so they get SKIPPED during large-scale changes and cannot be taught to newcomers.

AND FIDELITY COMPOUNDS . paths grow multiplicatively with each node. and low-fidelity doubles multiply too: TWO doubles at 10% inaccuracy => 99% chance of a bug.

-> so: SMALLER IS BETTER EVEN HERE. a handful of large tests beats one enormous one. shrink the SYSTEM to shrink the test. CHAIN pairwise tests, persisting one test's output as the next's input.

FOUR PHASES OBTAIN a system under test . SEED data .

ACT . VERIFY.

THE SYSTEM UNDER TEST, judged on TWO FACTORS THAT CONFLICT
HERMETICITY isolation from everything but this test.
FIDELITY resemblance to production.
forms: SINGLE-PROCESS (least faithful topology, can
be small) . SINGLE-MACHINE . MULTI-MACHINE . SHARED
(staging/production) . HYBRID.

-> testing IN production is TOO LATE to block the
release. reserving staging windows does not scale.

SHRINK IT AT THE PAINFUL BOUNDARIES

UI/BACKEND -- look-and-feel churn breaks tests
without behaviour changing; async is hard.
split at the backend's public interface.

THIRD PARTIES -- often no shared test environment,
and calls may cost. DO NOT let automated tests
call them. an important seam.

RECORD-REPLAY -- a big test generates traffic logs;
a small one replays them. matching arguments to
choose a response MAKES IT VERY LIKE STUBBING.

DATA SEEDED (state at the start) + TRAFFIC (sent during).
seeding is orders of magnitude harder: DOMAIN DATA
(binaries fail to start without it) . a REALISTIC
BASELINE (quality and quantity, or nobody believes
it) . SEEDING INTERFACES (writing straight to the
store BYPASSES the real triggers and checks).
sources: HAND-CRAFTED . COPIED . SAMPLED.

VERIFY MANUAL (does NOT scale sublinearly) . ASSERTIONS .
DIFFERENTIAL (no stated expectation -- A HUMAN must
judge every difference).

KEEPING THEM USABLE

FAST . cut scope or split in two. replace sleeps with
polling, events or subscriptions -- SLEEP-BASED
TESTS ALL FAIL WHEN THE FLEET IS OVERLOADED,
AND THAT SPIRALS. make internal timeouts tunable.

UNFLAKY . start by REDUCING SCOPE. beware: production
degrades gracefully rather than erroring, so a
flaky timeout LOOKS LIKE BROKEN CODE.

UNDERSTANDABLE . a stack trace is useless across
process boundaries. name the failure IN CONTEXT,
trace the call chain, and GIVE CONTACT DETAILS.

OWNED . documented owners, empowered and incentivised.
project lead for integration tests; a feature
owner for feature tests.

Q Name the five gaps, and say which one Part 3's fakes mostly close.

Q Which two properties of a system under test conflict, and what does each buy?

Q Give the three ways to verify a larger test and the distinct cost of each.

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
Standing up a hermetic system under test	Isolation from other tests and users, and the least exposure to concurrency and infrastructure flakiness.	Fidelity — it resembles production less — plus the work of building and maintaining the environment.	When you need the test to block a release, since anything shared or in production is available too late to do that.
Testing against a shared or production environment	The highest fidelity available and no environment to build — it already exists.	Conflicts with other users, waiting for code to be pushed there, and in production a real risk of user impact.	For monitoring-style checks of the live system. Never as the thing that gates the release.
Copying or sampling production data	Content fidelity — coverage of cases no author would have imagined, free of hand-crafted bias.	Volume that is hard to reason about, and any exposure risk in what you copied. Sampling trades coverage for tractability.	When hand-crafted data has left a fidelity gap. Smart sampling for the minimum data that keeps the coverage.
Differential comparison instead of assertions	Detection of unanticipated side effects without having to state expected behaviour in advance.	A human must judge every difference, noise must be actively suppressed, and you now maintain two systems.	When the space of failures is too large to enumerate — which is exactly where explicit assertions run out.
Splitting one enormous test into chained pairwise tests	Smaller systems under test, quicker runs, and failures that localise.	Plumbing to persist one test's output as the next one's input, and a journey no single test now covers end to end.	Whenever a user journey needs contributions from many internal systems. Smaller is better even here.
Polling or subscribing instead of sleeping	Tests that take the time the work takes, and that do not collapse when the fleet is loaded.	More code than a sleep, plus a timeout for the case where the state never arrives.	Always — and note the spiral: sleep-based tests fail under load, get rerun, and add more load.
Assigning a documented owner	Someone able to review changes and diagnose failures, which is what stops the test rotting.	A named person's time, on a test that spans other people's units.	Always. Ownership is listed as a precondition, not a nicety — an unowned larger test decays by default.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO UNDERSTAND IT	SAY IT THIS WAY TO SOUND LIKE AN EXPERT
“Our unit tests all pass, so we're fine to ship.”	“They ran in a vacuum. Nothing there checked the configuration, the load, or whether our doubles match the real service.”
“The outage was a config change, not a code bug.”	“Config is the number one cause of major outages. Is it in version control, and is it exercised by a larger test?”
“We mocked their service, so we've covered the integration.”	“We wrote that double without writing their service. Their contract could differ, and the double will go stale silently.”

“Let's make the end-to-end test cover everything.”	“Smaller is better even here. Shrink the system under test, or chain pairwise tests through persisted output.”
“We'll run it against production for realism.”	“Then it can't block the release — the system under test arrives too late. And a write there changes real state.”
“We'll book the staging environment on Tuesdays.”	“Reservation doesn't scale — users, services and conflicts all grow. We want a hermetic instance.”
“The UI test broke again after a redesign.”	“Look-and-feel churn breaks these without behaviour changing. Split at the backend's public interface.”
“We seeded the database directly, it was quicker.”	“That bypasses the triggers and checks the real binaries run. The state may not be reachable in production.”
“The test times out sometimes under load.”	“Sleeps fail when the fleet is busy, then get rerun, which loads it further. Poll, or subscribe to the event.”
“It just says assertion failed with a stack trace.”	“A stack trace is useless across process boundaries. Name what failed, trace the chain, and say who to contact.”

PART 4 · QUESTION 1

A logistics platform ships weekly. All 9,000 of its unit tests passed before a release that took the fleet offline for six hours. The cause was a routing table in a configuration file, checked into a separate repository with no review, whose format the binary had stopped accepting two releases earlier. In the same period: a partner's address-validation service began returning a new error code that the platform's hand-written double had never produced, so the handling path had never run; a promotional event drove twelve times normal traffic and the dispatch queue collapsed in a way no component exhibited alone; and a customer discovered that sorting by delivery window returned results in an order the platform had never specified but several integrators had come to depend on. The team's plan is to raise unit test coverage from 78% to 95%.

(a) Match each of the four incidents to the gap it demonstrates, naming the gap and its mechanism. **(b)** Two of the four have a named practice attached in this material. Give both practices and say precisely what each one prevents. **(c)** The sorting incident has a name. Give it, state what it says about the effective contract, and explain why more unit tests cannot close it. **(d)** Assess the coverage plan against the diagnosis, and say what the team should be building instead.

PART 4 · QUESTION 2

A team is asked to build automated coverage for a checkout journey spanning a web front end, a cart service, a pricing service, an inventory service and an external tax provider. Their design: one test that drives a real browser against the shared staging environment, calls the tax provider's live sandbox, seeds data by writing rows straight into the inventory database, and waits with a 30-second sleep between steps. It runs for 40 minutes, fails roughly a third of the time, and reports failures as “assertion failed” with a stack trace from the browser driver. It is scheduled after deployment to staging. Nobody is named as its owner; the three teams involved each assume one of the others maintains it.

(a) Name the two conflicting properties of a system under test, say where this design sits on each, and give the consequence of running it after deployment. **(b)** Identify the two boundaries at which this system should be cut, giving the specific reason for each. **(c)** Take the direct database seeding and the sleeps together: name what each one breaks, and give the replacement for the sleeps together with the failure mode they cause under load. **(d)** The failure output and the ownership arrangement are both defects. Give the three things a good larger-test failure must do, and state the rule about ownership and what happens without it.

2×2 — how faithful is the environment, and how isolated is it

COLUMNS: IS IT ISOLATED FROM EVERYTHING BUT THIS TEST? →

	Yes — hermetic, controlled by the test	No — shared with other tests, users or traffic
HIGH fidelity (resembles production)	What you are usually aiming for Real binaries, production-like configuration and topology, standing up and tearing down under the test's own control — so it can gate a release. <i>Move:</i> buy only as much of this corner as the risk needs. Shrink the system at the interface and third-party boundaries, and chain pairwise tests rather than growing one enormous one.	Truthful, and too late Staging or production: the highest fidelity available, nothing to build — and the code must already be there, so the test cannot block its own release. <i>Move:</i> use it for monitoring-style checks, and accept that anything it catches is already affecting users.
LOW fidelity (doubled, collapsed)	Fast, controlled, and quietly assuming A single-process or single-machine stand-up with doubles for the rest. Cheap and reliable — and every double is a belief about somebody else's contract. <i>Move:</i> inaccuracy compounds — two doubles at ten percent each put the odds of a bug near certainty. Prefer fakes owned by the interface's owners.	The worst of both Neither faithful nor isolated — a doubled system sharing an environment. Flaky for reasons that are not yours, untruthful about the ones that are. <i>Move:</i> pick a direction: take control of the environment, or drop the doubles and treat it as a monitoring check.

Read it as: the row is *fidelity*, the column is *hermeticity*, and they pull against each other — every step toward production buys truthfulness with reliability and control, and the top-left is the corner you construct rather than find. The bottom-right is where a test drifts when nobody owns it.

CARRY-OUT RULE FROM THIS PART

Write a larger test to put back a specific thing the vacuum removed, and make it as small as it can be while still putting that thing back.

Concept sketch — the whole subject, end to end

Four named groups, in the order the story runs: **THE SUITE** → **THE TEST** → **THE STAND-IN** → **THE VACUUM**. Every node carries a question. Cover the answers, walk the map, and each answer hands you the next question.

Legend: **bold** = a group heading **reversed = the one fact to remember** **Q** = retrieval cue

The four groups: **THE SUITE** — what it is for and how it is classified · **THE TEST** — writing one that never needs touching · **THE STAND-IN** — substituting a dependency · **THE VACUUM** — what none of it can see.

GROUP 1 - THE SUITE . it runs on trust, and trust is spendable

A TEST = one BEHAVIOUR . one INPUT . an observable OUTPUT .
a CONTROLLED ENVIRONMENT.

TWO PURPOSES: catch bugs, AND SUPPORT THE ABILITY TO CHANGE.
the more you want to change, the more you need fast tests.

TESTS DERIVE THEIR VALUE FROM THE TRUST ENGINEERS PLACE IN THEM. A BAD SUITE IS WORSE THAN NO SUITE.

Q what does that make speed and determinism -- taste, or a requirement?

TWO AXES SIZE = resources allowed (small: one process, NO sleep/IO/blocking . medium: one machine . large: anywhere).
SCOPE = how much code is VALIDATED, not executed (narrow / medium / broad).

Q give a broad-but-small test and a narrow-but-medium one.

THE MIX ~80 / 15 / 5. ICE CREAM CONE = mostly end-to-end.
HOURLASS = no middle, caused by TIGHT COUPLING.
larger tests are SANITY CHECKS, not the primary bug catcher.

EVERY TEST HERMETIC and OBVIOUS UPON INSPECTION -- because
THERE ARE NO TESTS FOR THE TESTS. so: no loops, no conditionals.

FLAKINESS IS ARITHMETIC $0.1\% \times 10,000/\text{day} = 10$ investigations.
NEAR 1% THE SUITE STOPS BEING BELIEVED. reruns only delay the root cause.

TEST WHAT YOU DO NOT WANT TO BREAK -- including how it FAILS.
if someone's change passes your tests and breaks you, that is YOUR test to write.

COVERAGE records that a line RAN, not that anything CHECKED it.
and as a target the floor becomes a CEILING.

HOW SUITES DIE BRITTLE (unrelated changes break it; teams stop refactoring) and SLOW (sleep() buried in a shared utility).
-> POLL at microsecond frequency. TREAT TESTS LIKE PRODUCTION CODE.

THE EDGE human judgement, and EXPLORATORY TESTING -- the target is UNKNOWN at the start. whatever it finds, AUTOMATE AFTER.

GROUP 2 - THE TEST . make it contract only what you promised

THE IDEAL TEST NEVER CHANGES unless the REQUIREMENTS change.

TWO FAILURES BRITTLE (breaks on an unrelated change; vs FLAKY = breaks with NO production change) and UNCLEAR (worse than unclear production code -- deleting a test proves nothing, so obscure ones get deleted having given zero value).

FOUR KINDS OF CHANGE refactoring / new feature / bug fix -> touch NO existing test. BEHAVIOUR CHANGE -> the only one, and the dearest, because users rely on the old behaviour.

Q a refactor broke a test. what are the two possible causes?

AGAINST BRITTLINESS

TEST VIA PUBLIC APIS -- then a broken test means a possibly broken USER. "public" = exposed outside the owning team, not the language keyword.

TEST STATE, NOT INTERACTIONS -- a call check passes if the record is deleted after, and fails if an equivalent call replaces it.

AGAINST UNCLARITY

COMPLETE AND CONCISE (both) . BEHAVIOURS NOT METHODS
(given/when/then; many-to-many) . NAME IT AFTER THE
BEHAVIOUR (needing "and" = two tests) . NO LOGIC (one
concatenation hid a doubled slash) . FAILURE MESSAGES that
carry expected, actual and parameters.

DESCRIPTIVE OVER DEDUPLICATED tests are MEANT to be stable, so
deduplication buys less; and they have no suite of their
own, so complexity costs more.

shared VALUES -> hide why. shared SETUP -> state the value
if you care. shared HELPERS -> a catch-all validate() kills
intent. TEST INFRASTRUCTURE NEEDS ITS OWN TESTS.

GROUP 3 - THE STAND-IN . every rung down spends truthfulness

SEAMS make substitution possible. DEPENDENCY INJECTION is the
usual one. TESTABILITY IS AN UPFRONT INVESTMENT.

0 REAL IMPLEMENTATION -- first choice, highest fidelity. use it
while FAST + DETERMINISTIC + SIMPLE DEPS. its bug failing
your test IS THE TEST WORKING.

1 FAKE -- lightweight, behaves like the real thing, HOLDS STATE.
fidelity TO THE CONTRACT, from the TEST'S PERSPECTIVE.
owned by the team that owns the real thing; built at the
ROOT of what is untestable; AND IT NEEDS ITS OWN TESTS.

2 STUB -- hardcoded returns. UNCLEAR, BRITTLE, and WEAK (no
guarantee of the contract, no state). the tell: you are
mentally stepping through the code to see WHY.
right for: forcing a state, and errors nothing else can
trigger. few, and each tied to an assertion.

3 INTERACTION TEST -- validates the CALL, not the result.
overused => CHANGE-DETECTOR TESTS. only for STATE-CHANGING
functions; avoid overspecification; supplement with a
larger test that checks STATE.

Q what does a green test mean at each of the four rungs?

GROUP 4 - THE VACUUM . what the speed was bought with

FIDELITY = how far a test reflects REAL behaviour. THE PRIMARY
REASON LARGER TESTS EXIST. more fidelity = more cost, and in
production, more RISK.

FIVE GAPS UNFAITHFUL DOUBLES (the same engineer writes the unit
AND its doubles, did not write the doubled thing -- and doubles
GO STALE) . CONFIGURATION (THE NUMBER ONE CAUSE OF MAJOR
OUTAGES; version-control it) . LOAD (highly EMERGENT -- it
belongs to the system, not its members) . THE UNANTICIPATED
(HYRUM'S LAW: the real contract is ALL VISIBLE BEHAVIOUR) .
EMERGENT BEHAVIOUR.

THE PRICE larger tests break all three developer-friendly
properties -- RELIABLE, FAST, SCALABLE -- and add OWNERSHIP
(unowned tests ROT) and STANDARDISATION. low fidelity also
COMPOUNDS: two doubles at 10% inaccuracy => 99% chance of a
bug -- so SMALLER IS BETTER EVEN HERE. shrink the SYSTEM to
shrink the test, and CHAIN pairwise tests.

FOUR PHASES OBTAIN . SEED . ACT . VERIFY. the system under test
is judged on HERMETICITY vs FIDELITY, IN CONFLICT -- and
production is too LATE to block its own release. cut at the
UI/backend boundary and at THIRD PARTIES.

DATA SEEDED + TRAFFIC. domain data, a realistic baseline, seeding
interfaces (direct writes BYPASS real checks). hand-crafted /
copied / sampled.

VERIFY MANUAL (not sublinear) . ASSERTIONS . DIFFERENTIAL (judged
by a human).

KEEP THEM USABLE cut scope . POLL instead of sleeping (sleeps fail
under load, get rerun, load it further) . name the failure in
CONTEXT with contact details . GIVE IT AN OWNER.

Master 2x2 — the decision card you can carry to other problems

The two questions that survive when the vocabulary is stripped away: **is this test believed** — fast, deterministic, clear, and failing only on meaningful change — and **is it truthful**, meaning does a pass actually entitle you to say the behaviour works?

COLUMNS: DOES A PASS ENTITLE YOU TO CLAIM THE BEHAVIOUR WORKS? →

	Yes — real or faithful, and the outcome is observed	No — doubled, assumed, or only the attempt is checked
BELIEVED (fast, stable, clear, fails only on real change)	A test worth having People act on a red result, and a green one means something. This is the only cell in which the suite is an asset rather than an expense. <i>Move:</i> both properties decay, and they decay separately. Trust is lost to flakiness, slowness and brittleness; truth is lost when a double drifts from the thing it stands for. Guard each with its own practice.	Trusted, and quietly wrong Fast, stable, green — and green because a stand-in was told to say so, or because only the call was checked. The most dangerous cell, because nothing announces the problem. <i>Move:</i> ask what a pass claims. If it claims “if my assumption held” or “an attempt was made”, you owe a fake with its own tests, or a larger test that checks real state.
NOT BELIEVED (slow, flaky, obscure, or brittle)	Truthful, and ignored It exercises the real thing and would catch the real fault — and it takes forty minutes, fails a third of the time, and people rerun it until it goes green. <i>Move:</i> the truth is wasted until the trust is repaired. Cut the scope, replace sleeps with polling, make the failure name what failed, and give it an owner.	Pure cost Neither believed nor truthful: maintenance, investigation time, and a false sense that something is covered. <i>Move:</i> this is where a bad suite is worse than no suite. Delete or rebuild rather than repair — and note that the false assurance has probably also stopped somebody looking at the system by hand.

Read it as: the row is *trust* and the column is *truth*, and the whole subject is the claim that a suite needs both and loses them to different causes. The same pair separates a smoke alarm that people still respond to from one taped over, a survey with a good sample from one with a good response rate, and a monitoring dashboard that is watched from one that is merely accurate.

THE THUMB RULE

Keep the suite believed, keep it truthful, and know which one each practice is defending. Speed, determinism and clarity buy trust; real implementations, faithful fakes and larger tests buy truth — and neither one substitutes for the other.

Symptom → diagnosis → move

Cover the middle and right columns and work down the left.

Designing and running a suite

WHAT YOU OBSERVE	MOST LIKELY DIAGNOSIS	WHAT TO DO
People rerun the build until it passes.	Flakiness has crossed the threshold where the suite stops being believed.	Treat each flake as a defect. Reruns trade compute for engineer time and only postpone the root cause.
The suite runs nightly rather than per change.	It is too slow, so it has been demoted — and the less often it runs, the less it is worth.	Push behaviour down into small tests, isolate the large ones from the fast suite, and hunt for sleeps in shared utilities.
A single sleep in a helper adds minutes to every run.	A wait-and-check heuristic used in place of a real signal.	Poll for the state transition at a frequency closer to microseconds, with a timeout in case the state never arrives.
Most tests drive the whole system through its front door.	The ice cream cone: slow, unreliable, and no help in locating a fault.	Rebalance toward narrow tests. Larger ones are sanity checks, not the primary way to catch bugs.
Plenty of unit and end-to-end tests, nothing between.	The hourglass — usually caused by coupling that stops dependencies being instantiated in isolation.	Fix the coupling, then add medium-scoped tests; the missing layer is what would have caught those end-to-end failures sooner.
Coverage is quoted as proof of health and never moves.	A metric that became a target, so the floor has become a ceiling.	Ask about behaviours instead — what customers rely on, breaking changes in dependencies, stability. Measure coverage from small tests only.
Somebody else's change broke your product; your tests passed.	The behaviour was never tested, so nobody promised it.	Fix it and add the test. Anything you do not want to break needs a test on it.
A test contains a loop over several cases.	Logic in a test, which nothing else checks.	Split it into separate straight-line tests and tolerate the duplication.

Writing and maintaining a test

WHAT YOU OBSERVE	MOST LIKELY DIAGNOSIS	WHAT TO DO
A pure refactoring broke a batch of tests.	Either it was not a pure refactoring, or the tests were written at the wrong level of abstraction.	Move them onto the public surface. A test that breaks should imply a user who would break.
A method was made public purely so a test could reach it.	The test is contracting an internal that no user depends on.	Find the public path that exercises the behaviour — or accept that this is a helper and test it through its callers.
A test verifies that a save call happened.	Interaction testing where state testing was available.	Read the state back instead. The call check passes even if the record is removed immediately afterwards.
One test method has grown to assert five outcomes.	Tests framed around methods rather than behaviours; it was extended each time the method grew.	Split into one test per behaviour, structured given/when/then, each named after what it guarantees.
A test name needs the word “and”.	It covers more than one behaviour.	Write two tests. The name is a scope check as well as a label.
The expected value is computed by the test.	Logic concealing a possible bug — the test and the code can be wrong in the same way.	Write the expected value out in full. Duplication is the cheaper of the two problems.

A reader cannot tell where a value in a test came from.	Shared setup or shared constants have cost the test its completeness.	State the value in the test body, overriding the default, whenever the test actually cares about it.
Every test ends with the same validate helper.	A catch-all assertion block that hides each test's intent and makes one bug fail many tests.	Assert one conceptual fact per helper, and let each test say what it is for.
The failure report says only that an assertion failed.	The message carries none of the diagnosis.	Report the desired outcome, the actual outcome and the relevant parameters — ideally enough to diagnose without opening the test.

Substituting a dependency, and going larger

WHAT YOU OBSERVE	MOST LIKELY DIAGNOSIS	WHAT TO DO
The real dependency is awkward to construct, so it was doubled.	A construction problem being solved with a fidelity sacrifice.	Use production's own factory or dependency injection. Construction cost is not an argument about truthfulness.
Eight canned return values precede the assertion.	Overused stubbing — and you are stepping through the system mentally to see why each is there.	Use a fake or the real implementation. Keep stubs few, and tie each to an assertion.
The suite passed while production silently failed.	A stub encoded a belief about the dependency's contract that nothing verified.	Move to something that honours the contract — and remember stubs cannot hold state, so you could not have read the result back.
Renaming an internal call failed a hundred tests.	Change-detector tests: they assert the route, so any production change fails them.	Assert state. Reserve call checks for state-changing functions and for cases where call count or order is the behaviour.
No fake exists for a service you must double.	The owning team may not know the concept or its benefit.	Ask them first; then write your own by wrapping the calls in one class; then fall back to the real implementation or another technique.
A fake has drifted from the service it stands for.	It has no tests, so its behaviour diverged as the real implementation evolved.	Write tests against the public interface and run them against both the fake and the real thing.
An outage was caused by a configuration file.	The gap unit tests structurally cannot cover — and the leading cause of major outages.	Put configuration in version control, and add a larger test that starts the binary against it.
Integrators depend on behaviour you never specified.	The effective contract is all visible behaviour, not the declared interface.	Differential comparison against the previous version, since the failure space is too large to enumerate as assertions.
The end-to-end test needs the whole estate stood up.	The test's scope is following the system's scope.	Shrink the system: cut at the interface boundary and at third parties, and chain smaller pairwise tests through persisted output.
A test runs only against staging after deployment.	The system under test arrives too late to block its own release.	Stand up a hermetic instance under the test's control if it is meant to gate anything.
Nobody knows who maintains a large test.	It spans several units, so it spans several owners — and unowned tests rot.	Name an owner: the project lead for integration tests, a feature owner for feature tests, empowered and incentivised to maintain it.

Reverse mapping — you see a practice, what is it there for?

The other direction. Use this when auditing a suite rather than writing one test: for each thing you find, say what it was buying and what it does *not* buy.

WHAT YOU FIND PEOPLE DOING	WHAT IT COUNTERS	WHAT IT DOES <i>NOT</i> COVER
Forbidding I/O and blocking calls in small tests	The two biggest sources of slowness and nondeterminism, which are what erode belief in a suite.	Scope. A small test can still validate a great deal of code, and still fail to localise a fault.
Classifying every test by size	Suites that silently grow slow, and arguments in review conducted in the wrong vocabulary.	Whether the test is any good. Size constrains resources, not quality or relevance.
Aiming at a mostly-narrow mix	Slow, unreliable suites and failures that start diagnosis from the whole system.	Contracts between components, which narrow tests cannot verify — that is the middle layer's job.
Testing through the public surface	Brittleness: a broken test now implies a user who would break.	Flakiness, and unclarity. A public-surface test can still be nondeterministic or unreadable.
Asserting on state rather than calls	Tests that pass when the result is undone, and fail when an equivalent route is substituted.	Cases where the call itself is the behaviour — a cache that must not re-read, an email that must be sent once.
One test per behaviour, named after it	Test methods that grow with the method under test until nobody can say what they cover.	Brittleness from the wrong entry point. A well-named behaviour test can still poke at internals.
Writing expected values out in full	Logic that can be wrong in the same way as the code, hiding the bug it was meant to catch.	Maintenance. You will update those literals by hand when the requirement genuinely changes.
A fake maintained by the interface's owners	Doubles that encode one engineer's guess at somebody else's contract, and go stale silently.	Anything you are explicitly measuring, such as latency — you cannot fake what the test is about.
Contract tests run against both the fake and the real thing	Divergence over time as the real implementation evolves and the fake does not.	Speed for everyone else. The real-implementation runs are slow, and only the fake's owners need them.
Restricting call checks to state-changing functions	Redundant assertions on reads, which add brittleness without adding correctness.	The underlying weakness: even on a write, the check says the attempt was made, not that it worked.
Keeping configuration in version control	The leading cause of major outages appearing as unattributable flakiness.	Correctness of the configuration itself, which still needs a test that starts the binary against it.
Recording real traffic and replaying it	Hand-crafted data's narrowness and its author's biases, and the cost of calling third parties.	New behaviour. When the client changes enough, requests stop matching and the recording must be regenerated.
A hermetic system under test	Conflicts with other users, waiting for a deployment, and the flakiness of shared environments.	Fidelity. It resembles production less than production does, which is the whole reason larger tests exist.
Differential comparison between versions	Unanticipated side effects, where the failure space is too large to write down as assertions.	Judgement. Nobody stated what should happen, so a human must decide whether each difference is intended.
Naming an owner for every large test	Rot: tests nobody can modify, and failures nobody resolves.	Standardisation. An owned test can still be built in a way no other team can run.

Numbers worth remembering

NUMBER	WHAT IT IS
4 elements	The simplest test: a single behaviour, a specific input, an observable output or behaviour, and a controlled environment such as a single isolated process.
2 purposes	Catching bugs, and supporting the ability to change. The second is stated as equally important, and it is the one that scales with how fast you want to move.
2 axes	Size — the resources a test may consume and what it is allowed to do. Scope — how much code it validates. Interrelated but distinct, and both needed.
3 sizes	Small runs in a single process, often a single thread, and may not sleep, perform I/O or make blocking calls. Medium runs on a single machine, may span processes and use threads, and may not reach another machine. Large may run wherever it wants.
3 scopes	Narrow validates a single class or method; medium validates interactions between a small number of components; broad validates several distinct parts or emergent behaviours no single method expresses.
80 / 15 / 5	The rough target mix by test case count: narrow-scoped unit tests validating most business logic, medium-scoped integration tests validating interactions, and end-to-end tests validating the whole system. Every team's mix differs.
2 antipatterns	The ice cream cone — many end-to-end tests and few others, typical of prototypes rushed to production. The hourglass — many end-to-end and many unit tests but few in between, caused by tight coupling that stops dependencies being instantiated in isolation.
$0.1\% \times 10,000 = 10$	The arithmetic of flakiness: a one-in-a-thousand chance of a spurious failure, across ten thousand daily test runs, is ten investigations a day.
~1%	The flakiness level at which tests begin to lose value, because engineers stop reacting to failures — at which point any value the suite provided is eliminated.
90 of 100	The definition of code coverage: if your tests execute ninety of a hundred lines you have ninety percent coverage — which records that a line was invoked, not what happened as a result.
80% as a ceiling	The failure mode of a coverage target: a bar set as a floor is treated as a ceiling, and changes start landing at exactly the required number.
4 kinds of change	Pure refactorings, new features and bug fixes should require no changes to existing tests. Behaviour changes are the one case that should — and they are significantly more expensive, because users rely on the current behaviour.
3 rules of thumb	Scope of a unit is more art than science. A class supporting only one or two others <i>probably</i> is not its own unit and should be tested through them; anything usable without consulting its owners <i>almost certainly</i> is; and a general-purpose support library is too, redundancy included.
2 failure directions	How an interaction test fails: it passes when a bug undoes the recorded action shortly afterwards, and it fails when the system is refactored to achieve the same result by a different call.
3 parts of a behaviour	Given — how the system is set up. When — the action taken. Then — the result validated. Also expressed as arrange, act, assert.
3 levels of granularity	How a well-structured test is read: the method name for a rough description, the given/when/then for a formal one, and the code for exactly how the behaviour is expressed.
3 impacts	What substituting a dependency introduces: testability (the code must permit swapping), applicability (misuse produces brittle, complex, less effective tests), and fidelity (how closely the double resembles the real thing).
3 techniques	Faking — a lightweight working implementation. Stubbing — hardcoding what a function returns. Interaction testing — validating how a function was called without executing it.
3 dangers of stubbing	Tests become unclear, tests become brittle, and tests become less effective — the last because nothing guarantees the stubbed behaviour matches the contract, and because stubs cannot store state.

2 categories of call	State-changing functions have side effects outside the system under test and may reasonably be interaction-tested. Non-state-changing functions return information and change nothing, so verifying the call is usually redundant.
2 warranted cases	When interaction testing is justified: when state testing is impossible because neither a real implementation nor a fake can be used, and when the number or order of calls would itself cause undesired behaviour.
15 minutes or 1 hour	The default timeouts for large tests — with some running for multiple hours or even days.
5 gaps	What unit tests cannot cover: unfaithful doubles, configuration issues, behaviour under load, unanticipated behaviours and inputs, and emergent behaviour — the vacuum effect.
The number one cause	Configuration changes, as the leading source of major outages — written in different languages from production code, rolled out faster, and harder to test.
3 properties	What a developer-friendly test must be: reliable, fast, and scalable. Larger tests often violate all three, and add ownership and standardisation problems on top.
$2 \times 10\% \rightarrow 99\%$	How low fidelity compounds: with two service doubles that are each ten percent inaccurate, the likelihood of a bug when the system is assembled is ninety-nine percent.
4 phases	The workflow of a large test: obtain a system under test, seed the necessary data, perform actions, verify the behaviours.
2 conflicting factors	Hermeticity — isolation from anything but the test in question. Fidelity — resemblance to the production system. These are often in direct conflict.
5 forms	The system under test may be single-process, single-machine, multi-machine, a shared environment such as staging or production, or a hybrid.
2 kinds of data, 3 sources	Seeded data sets the state at the start; test traffic is sent during execution. Data may be hand-crafted, copied from production, or sampled — smart sampling copying the minimum needed for maximum coverage.
3 verifications	Manual, which does not scale sublinearly; assertions, which state the intended behaviour explicitly; and differential comparison, where nothing is stated and a human must judge each difference.
3 requirements of a failure	A good larger-test failure identifies clearly what failed, minimises the effort to find the root cause across process boundaries, and provides support and contact information.

Glossary

TERM	MEANING
Test suite	Hundreds or thousands of simple tests taken in aggregate, telling you how well a product conforms to its intended design and when it does not.
Size	The resources a test may consume and what it is allowed to do — small, medium or large — determined by how it runs rather than by its length.
Scope	How much code a test validates, as distinct from how much code it executes. Narrow, medium or broad.
Hermetic	A test containing all the information needed to set up, execute and tear down its environment, assuming as little as possible about the outside world, including test ordering.
Flaky	A test that fails nondeterministically without any change to production code — distinct from a brittle test, which fails because of an unrelated change that introduced no bug.
Ice cream cone	A suite of many end-to-end tests and few integration or unit tests: slow, unreliable and difficult to work with.
Hourglass	A suite of many end-to-end and many unit tests with few integration tests, produced by coupling that prevents dependencies being instantiated in isolation.
Code coverage	A measure of which lines of feature code are exercised by which tests. It records invocation, not verification, and becomes a ceiling once it is a target.
Brittle test	One that fails in response to an unrelated change to production code that introduced no real bug.

Unchanging test	The ideal: a test that never needs to change after it is written unless the requirements of the system under test change.
Public API	In this context, the surface a unit exposes to third parties outside the team that owns the code — not necessarily what a language's visibility keywords call public.
State testing	Observing the system itself after invoking it, to see what it has become.
Interaction testing	Checking that the system took an expected sequence of actions on its collaborators, without executing the implementation of those actions.
Behaviour	Any guarantee a system makes about how it will respond to a series of inputs while in a particular state — expressible as given, when, then.
Complete and concise	Complete: the body contains everything a reader needs to understand how the result is reached. Concise: it contains nothing else.
Descriptive over deduplicated	The preference in test code for descriptive and meaningful phrasing over minimal repetition, because tests are meant to be stable and have no suite of their own.
Test infrastructure	Code shared across multiple test suites. Closer to production code than to test code — many callers, hard to change safely, and it must have its own tests.
Test double	An object or function that stands in for a real implementation in a test.
Seam	A way to make code testable by allowing different dependencies in the test than in production — commonly created by dependency injection.
Fake	A lightweight implementation of an interface that behaves similarly to the real one but is unsuitable for production, such as an in-memory database. It holds state, and it needs its own tests.
Stub	A function given hardcoded behaviour by the test — no contract, no state, and no guarantee that the value it returns is one the real implementation could produce.
Change-detector test	A test that fails in response to any change to production code even when the behaviour is unchanged — the result of overusing interaction testing.
Fidelity	How closely a double, or a test environment, reflects the real behaviour of the thing it stands for. Increasing it raises cost and, in production, risk.
Classical / mockist	Two styles: preferring real implementations in tests, versus preferring mocking frameworks. The first is the one that scales.
System under test	The thing a larger test stands up and exercises. Judged on hermeticity and fidelity, which are often in direct conflict.
Hermeticity	A system under test's isolation from use and interaction other than the test in question — the property that removes concurrency and infrastructure flakiness.
Seeded data / test traffic	Data preinitialised into a system to set its state at the start of a test, versus data sent to it by the test during execution.
Differential comparison	Running two versions, sending identical data and comparing the outputs. Nothing is stated as expected, so a human must judge every difference.
Record and replay	Using a larger test to record traffic to external services, and a smaller test to replay it — matching arguments to choose a response, which makes it very like stubbing.
Exploratory testing	Creative manual testing in which the specific problems to be found are unknown at the start — and whose findings should be converted into automated tests.

Answer key

PART 1 · Q1 — THE 55-MINUTE SUITE

(a) The typical test is **broad in scope** — it validates the interaction of several distinct parts rather than the logic of one class or method — and **large in size**, since it runs a real database and a real message broker as separate processes and is not confined to a single machine. The suite's shape is the **ice cream cone**: many end-to-end tests and few integration or unit tests. Two properties attach to it: such suites **tend to be slow, unreliable and difficult to work with**, and the pattern **often appears in projects that start as prototypes and are quickly rushed to production, never stopping to address testing debt**. (b) Forty spurious failures in four thousand tests is **1% flakiness**, which is exactly the stated danger line: **as you approach 1% flakiness the tests begin to lose value**, because **it does not take needing to investigate many flakes before a team loses trust in the test suite — after which engineers stop reacting to test failures, eliminating any value the suite provided**. The rerun rule is a legitimate technique misapplied. Rerunning failures automatically is **effectively trading processor cycles for engineering time, and at low levels of flakiness that trade-off makes sense** — but this is not a low level, and in any case **rerunning a test is only delaying the need to address the root cause**. Worse, a rule that merges on either green is the operational definition of no longer reacting to failures at all. (c) First, coverage **measures only that a line was invoked, not what happened as a result** — it is possible to exercise a lot of lines with a few tests while never checking that each line does anything useful. The figure here is additionally inflated, because coverage should be **measured from small tests only, to avoid the coverage inflation that occurs when executing larger tests**, and this suite is almost entirely large. Second, like other metrics **it quickly becomes a goal unto itself**: a bar set at 80% is treated **as a ceiling rather than a floor**, which is precisely why the number has not moved in a year. What should be asked instead is behavioural: **do you have confidence that everything your customers expect to work will work; do you feel confident you can catch breaking changes in your dependencies; are your tests stable and reliable?** Trying to answer “do we have enough tests?” with a single number **ignores a lot of context and is unlikely to be useful**. (d) Responsibility sits with the payments team. The rule is to **test everything that you don't want to break** — if you want confidence that a system exhibits a particular behaviour, the only way to be sure is to write an automated test for it — and it is stated in exactly this situation: **if unrelated infrastructure changes pass all of your tests but still break your team's product, you are on the hook for fixing it and adding the additional tests**. So the team owes the fix and the missing test on the date-parsing behaviour it was relying on, rather than a request for more care from the library team.

PART 1 · Q2 — THE SLEEPING HELPER AND THE REVIEWER

(a) Six hundred tests at 400 milliseconds each is **240 seconds — four minutes — added to every single run of the suite**. The antipattern is unnecessary speed limits introduced by sleep and timeout functions, which are **often used as naive heuristics before checking the result of nondeterministic behaviour**: half a second here or there does not seem dangerous, but **embedded in a widely used utility it soon adds minutes of idle time to every run**. The recommended replacement is to **actively poll for the state transition with a frequency closer to microseconds**, and the safeguard is to **combine that with a timeout value in case the test fails to reach a stable state**. (b) Both objections are wrong, and they are wrong in opposite directions. The date-conversion test is **narrow in scope** — it validates one method — but it cannot be **small**, because a small test **must run in a single process and may not perform I/O**, and reading a 200 MB offset file from disk is I/O; it is **medium in size** and narrow in scope. The nightly-report test is the mirror image: **broad in scope**, since it covers ingest, aggregation and formatting together, yet **small in size**, because everything out-of-process has been replaced by an in-process stand-in and the whole thing runs in one process without I/O or blocking calls. The general point is the one the reviewer is missing: **size and scope are interrelated but distinct concepts**. Narrow tests *tend* to be small and broad tests *tend* to be medium or large, **but this is not always the case** — and “unit test” names a scope, not a size. (c) The researcher performed **exploratory testing: a fundamentally creative endeavour in which someone treats the application under test as a puzzle to be broken**, perhaps by executing an unexpected set of steps or by inserting unexpected data. Its defining property is that **the specific problems to be found are unknown at the start** — they are gradually uncovered by probing commonly overlooked code paths or unusual responses. The omitted step is the one that makes it pay twice: **as soon as an exploratory test discovers an issue, an automated test should be added to prevent future regressions**, so that a flaw found once by a human can then be checked continuously and at scale. (d) The target is roughly **80% narrow-scoped unit tests** validating the majority of the business logic, **15% medium-scoped integration tests** validating interactions between two or more components, and **5% end-to-end tests** validating the entire system — by test case count, with every team's mix a little different. The reasoning: unit tests form an excellent base because they are **fast, stable, dramatically narrow the scope and reduce the cognitive load required to identify all the possible behaviours a class or function has**, and because they **make failure diagnosis quick and painless**; the recommended mix is determined by two goals, **engineering productivity and product confidence**, and favouring unit tests gives high confidence quickly and early in development. What the largest tests should not be relied on for is stated plainly: **larger tests act as sanity checks as the product develops; they should not be viewed as a primary method for catching bugs**.

PART 2 · Q1 — NINETEEN BREAKAGES AND ONE SHIPPED BUG

(a) The nineteen breakages are **brittleness**: a brittle test is one that **fails in the face of an unrelated change to production code that does not introduce any real bug**. Its look-alike is a **flaky** test, which fails **nondeterministically, without any change to production code at all** — same loss of trust, different cause and different cure. The change described is a **pure refactoring**: the internals changed and the interface did not. Since **tests should not need to change during a pure refactoring**, and their whole role in that case is to ensure the refactoring did not change behaviour, breakage proves one of exactly two things — **either the change is affecting the system's behaviour and is not a pure refactoring, or the tests were not written at an appropriate level of abstraction**. Here the interface was untouched, so it is the second. (b) The practice is to **test via public APIs**: write tests that **invoke the system being tested in the same way its users would**, calling its public surface rather than its implementation details. The argument is definitional — **if tests work the same way as the system's users, then by definition a change that breaks a test might also break a user**; such tests **form explicit contracts**, and testing only those contracts means you are **free to do whatever internal refactoring you want without having to worry about making tedious changes to tests**. “Public” in this context means **the API exposed by that unit to third parties outside of the team that owns the code, which does not always align with the notion of visibility provided by some programming languages**. (c) The test verifies that a specific call was made against a collaborator, and such tests fail in two directions. In the first, **a bug leaves the system in the wrong state even though the call was made, so the test passes even though we would have wanted it to fail** — which is exactly the shipped bug: `append()` was called once with the right argument, the assertion was satisfied, and nothing was ever made durable, because verifying a call says nothing about what the call resulted in. In the second, **the system is refactored to call a slightly different API to write an equivalent record, and the test fails even though we would have wanted it to pass**. The alternative is to **test directly against the state of the system**: perform the action and then read the state back, which **more accurately expresses what we care about — the state of the system under test after interacting with it**. (d) The proposed rule entrenches the defect. It treats the internal structure as something the tests are entitled to depend on, which is precisely what makes them brittle, and it accepts a permanent tax on refactoring — over time that friction **makes a team reticent to perform necessary refactoring to keep a codebase healthy**. It also concedes the automation: **if a set of tests needs to be manually tweaked by engineers for each change, calling it an “automated test suite” is a bit of a stretch**. The correct move is to rewrite the tests against the public surface and to assert on resulting state — after which the array swap, the inlining and the handle change would have broken nothing, and the uncommitted-reservation bug would have failed a test, because reading the ledger back would have found it empty.

PART 2 · Q2 — THE SIXTY-LINE TEST

(a) The mistake is framing tests around methods — matching **the structure of the tests to the structure of the code, such that every production method has a corresponding test method**. It is convenient at first and degrades: **as the method being tested grows more complex, its test also grows in complexity and becomes more difficult to reason about**, and it **naturally encourages unclear tests because a single method often does a few different things under the hood and might have several tricky edge and corner cases**. The alternative unit is the **behaviour**: **any guarantee that a system makes about how it will respond to a series of inputs while in a particular state**, expressible as given, when and then — and the mapping between methods and behaviours is **many-to-many**. Three reasons behaviour-driven tests read better: they **read more like natural language, allowing them to be naturally understood rather than requiring laborious mental parsing**; they **more clearly express cause and effect because each test is more limited in scope**; and because each is short and descriptive, they **make it easier to see what functionality is already tested and encourage engineers to add new streamlined test methods instead of piling onto existing ones**. (b) The catch-all helper damages **intent and localisation**: tests using a fixed validation strategy are **less behaviour driven**, it becomes **much more difficult to determine the intent of any particular test and to infer what exact case the author had in mind**, and when bugs are introduced such helpers **make them more difficult to localise because they will frequently cause a large number of tests to start failing**. The corrective rule: **the best validation helper methods assert a single conceptual fact about their inputs**, and earn their place when that condition is conceptually simple but requires looping or conditional logic that would reduce clarity inside the test body. The setup values damage **completeness**: a test that depends on a particular value defined in setup **seems incomplete because a reader needs to go hunting to discover where it came from**. The corrective rule: **tests that explicitly care about particular values should state those values directly, overriding the default defined in the setup method if need be** — the result contains slightly more repetition and is far more descriptive and meaningful. (c) The test builds its expectation the same way the code builds the result, so **both can be wrong in the same way and the test cannot detect it**. The rule broken is **don't put logic in tests**: clear tests are **trivially correct upon inspection**, and logic — operators, loops, conditionals — means **you need to do a bit of mental computation to determine the result instead of just reading it off the screen**; the worked case is a single string concatenation that concealed a doubled separator until the value was written out in full. And **if you feel like you need to write a test to verify your test, something has gone wrong**. What should be tolerated instead is duplication: writing the whole expected value out, since duplicating it is **a small price to pay for making the test more descriptive and meaningful — stick to straight-line code over clever logic**. (d) A good failure message **clearly expresses the desired outcome, the actual outcome, and any relevant parameters**. A rewrite: *“Expected a cart total of 1104 after applying the 15% discount to an item priced 1299, but the total was 1299.”* Note why the generic form arises: an assertion that receives only a boolean value **can only give a generic message such as “expected true but was false”**, whereas one that explicitly receives the subject of the assertion can report what the value actually was.

PART 3 · Q1 — THE NOTIFICATIONS SERVICE

(a) Two techniques are in use. **Stubbing** — giving behaviour to a function that otherwise has no behaviour of its own, by specifying exactly what values it returns — supplies the six canned values. **Interaction testing** — validating how a function is called without actually calling the implementation of the function — supplies the two verifications. A green test claims correspondingly less at each. With stubbing, it claims the system behaved correctly *on the assumption that the author's beliefs about the dependencies were accurate*, because **with stubbing there is no way to ensure the function being stubbed behaves like the real implementation**. With interaction testing, it claims only that certain functions were called as expected: **it cannot tell you that the system under test is working properly**, and it **requires you to make an assumption about the behaviour of the code**. (b) The 140 failures are the signature of **change-detector tests** — tests that overuse interaction testing and therefore **fail in response to any change to the production code, even if the behaviour of the system under test remains unchanged**. What is being asserted instead of the intended behaviour is **implementation detail**: to validate that a function was called, **you are exposing to the test that the system under test calls this function**, which leaks implementation details of production code into the tests and is what makes them brittle. (c) Two failures compound. The interaction check on the delivery call proves the call happened, and the primary issue with interaction testing is precisely that **it can only validate that certain functions are called** — it cannot establish that the send succeeded. And the stubbing hardcodes part of the contract: every canned receipt sets the accepted flag true, which is the danger that **tests become less effective** — **you will be forced to duplicate the details of the contract, and there is no way to guarantee that the contract is correct**, that is, that the stubbed function has fidelity to the real implementation. There is a further consequence here: **with stubbing there is no way to store state**, so the test could not have queried the delivery client for what had actually been recorded even if it had wanted to. (d) Four moves, in order. **First, ask the owners of the API to create a fake** — they may not be familiar with the concept, or may not realise the benefit it provides to users of their API. **Second, if they are unwilling or unable, write your own**, which can be done by **wrapping all calls to the API in a single class and then creating a fake version of that class**; this is often much simpler than faking the entire API, since you generally need only a subset of its behaviour. **Third, settle on using a real implementation** and accept its trade-offs. **Fourth, resort to other test double techniques** and accept theirs. On ownership and placement: **the team that owns the real implementation should write and maintain the fake**, because whenever the behaviour of the real implementation changes the fake must be updated to match; and to limit how many exist, **a fake should typically be created only at the root of the code that isn't feasible for use in tests** — for a database, at the database API itself rather than for each class that calls it.

PART 3 · Q2 — FOUR DEPENDENCIES IN A BOOKING ENGINE

(a) **The money type**: use the real one. **A real implementation is preferred if it is fast, deterministic and has simple dependencies**, and **a real implementation should be used for a value object** — an amount of money, a date, an address, a collection class. “Doubles are the standard here” is not a criterion. **The pricing library**: keep it real. The arithmetic is 1.2 milliseconds across 900 cases, which is **1.08 seconds added to the whole suite**. There is **no exact answer** on how slow is too slow — it depends on whether engineers feel a loss in productivity and on how many tests use the implementation, since **one second extra per test case may be reasonable if there are five test cases but not if there are 500** — and for borderline situations **it is often simpler to use a real implementation until it becomes too slow to use, at which point the tests can be updated**. Roughly a second across the whole suite is not that point. (b) The seat allocator is nondeterministic: **a test is nondeterministic if its outcome can change even when the system under test remains unchanged**, and nondeterminism leads to flakiness. Two causes apply: **a more complex real implementation is likelier to be nondeterministic**, for instance where output depends on thread execution order, and **code that relies on the system clock**, whose output differs with the current time; the fix for the latter is that **instead of relying on the system clock, a test can use a test double that hardcodes a specific time**. The test for whether flakiness warrants a response: **if use of a real implementation rarely causes flakiness it might not warrant a response, because there is little disruption to engineers — but if flakiness happens often, it might be time to replace the real implementation with a test double**. (c) The technique is **stubbing**, which is appropriate here for the reason that distinguishes it: because **a function's behaviour is defined inline in the test**, stubbing can simulate a wide variety of return values or errors that might not be possible to trigger from a real implementation or a fake. Two constraints keep it honest: **each stubbed function should have a direct relationship with the test's assertions**, so its purpose is clear; and **a test should typically stub out a small number of functions**, because stubbing many leads to less clear tests and **can be a sign that stubbing is being overused, or that the system under test is too complex and should be refactored**. So keep the fake for everything else and stub only the declined-after-authorisation path. On the fake itself: **a fake might not need to have 100% of the functionality of its corresponding real implementation, especially if such behaviour is not needed by most tests — for example, error handling code for rare edge cases** — and in that case **it is best to have the fake fail fast**, raising an error if an unsupported code path is executed, which **communicates to the engineer that the fake is not appropriate in this situation**. (d) The technique is **interaction testing**, justified by the second warranted case: **differences in the number or order of calls to a function would cause undesired behaviour**, and it is useful **because it could be difficult to validate this behaviour with state testing**. That is the situation exactly: the provider is external, so there is no state to read back, and the only observable difference between working and broken is the number of calls. It sits inside the general restriction rather than at its edge — **perform interaction testing only for state-changing functions**, those with side effects on the world outside the system under test, and dispatching a confirmation is one, whereas for **non-state-changing** functions the check is **usually redundant, given that the system**

under test will use the return value to do other work that you can assert. And when writing it, **avoid overspecification** — verify the minimum necessary for the behaviour in question.

PART 4 · Q1 — FOUR INCIDENTS AND A COVERAGE PLAN

(a) **The routing table: configuration issues.** Unit tests cover code within a binary, **but that binary is typically not completely self-sufficient in terms of how it is executed** — there is usually a deployment configuration or starter script, and production instances have their own configuration files; **issues with these files, or with the compatibility between the state they define and the binary, can lead to major user issues, and unit tests alone cannot verify this compatibility.** The partner's new error code: **unfaithful doubles.** When a heavyweight dependency is replaced **it becomes possible that the replacement and the doubled thing do not agree:** the engineer writing the test defines the double, and **usually did not write the thing being doubled and can be misinformed about its actual behaviour,** so **the understanding of the contract is invalid.** Worse, **doubles become stale** — if the double is invisible to the author of the real implementation and that implementation changes, **there is no signal that the test should be updated.** **The twelvefold traffic: issues under load.** Such testing requires sending large volumes of traffic, which does not fit a typical unit test — and **load and stress handling are highly emergent properties: complex behaviours that belong to the overall system but not the individual members,** which is why no component exhibited the collapse alone. **The sorting order: unanticipated behaviours.** Unit tests are **limited by the imagination of the engineer writing them — they can only test for anticipated behaviours and inputs,** while the issues users find are mostly unanticipated. (b) Two practices. For configuration: **keep it in version control as well as your code,** because then **changes to configuration can be identified as the source of bugs rather than introducing random external flakiness, and can be built into large tests** — precisely what a separate unreviewed repository prevents. For doubles: **if teams provide fakes for their own services, this concern is mostly alleviated,** since a fake maintained by the owning team tracks real behaviour instead of encoding a consumer's guess. (c) The sorting incident is an instance of **Hyrum's Law: even if we could test 100% for conformance to a strict specified contract, the effective user contract applies to all visible behaviours, not just a stated one,** and **it is unlikely that unit tests alone test for all visible behaviours not specified in the public API.** More unit tests cannot close it, because you can only write a test for a behaviour you have thought of — and these are by definition the ones nobody specified. The type of larger test that does address it is **differential comparison,** which detects unanticipated side effects without requiring anyone to state the expectation in advance. (d) The plan is aimed at the wrong axis: none of the four incidents is a defect in code logic inside a binary, which is what unit tests and coverage bear on — and coverage **only measures that a line was invoked, not what happened as a result.** What the diagnosis calls for is a set of larger tests, each putting back a different removed thing: **deployment configuration testing,** really a smoke test — **if the system under test starts successfully the test passes, and if not it fails;** a fake for the partner service, requested from its owners; **performance, load and stress testing built to look as close to production as possible;** and **differential regression testing** for the unanticipated — plus the configuration file into version control.

PART 4 · Q2 — THE CHECKOUT JOURNEY

(a) **Hermeticity — isolation from usages and interactions other than the test in question, giving the least exposure to concurrency and infrastructure flakiness** — and **fidelity, accuracy in reflecting the production system.** These two factors are often in **direct conflict.** This design sits at low hermeticity, since shared staging and a live external sandbox mean **the test might conflict with other simultaneous uses,** bought with high environmental fidelity. Scheduling it after deployment is decisive: such tests **cannot run until the code reaches that environment, so they cannot block its release — the system under test is too late.** (b) Two boundaries. **Between front end and back end:** user interface tests are **notoriously unreliable and costly,** since **interfaces often change in look-and-feel ways that make the tests brittle but do not actually impact the underlying behaviour** and **often have asynchronous behaviours that are difficult to test** — so where the back end provides a public interface, **split the tests there and use it to drive the end-to-end tests.** **The third-party boundary:** such systems **might not have a public shared environment for testing, and there can be a cost with sending traffic to a third party, so it is not recommended to have automated tests use a real third-party API — that dependency is an important seam at which to split tests.** (c) **Direct database seeding** breaks the seeding-interface point: **writing directly to a datastore might bypass triggers and checks performed by the actual binaries that perform the writes,** so the state may be one production could never produce. **The sleeps** break the speed and flakiness guidance: naive tests **use time-based sleeps to wait for nondeterministic action,** but **real production users want to wait as little as possible, so tests should react the way they would.** The replacements: **polling at a frequency closer to microseconds, with a timeout; an event handler; or a subscription to a notification system.** The failure mode under load is a spiral: **tests relying on sleeps all start failing when the fleet running them becomes overloaded, and that spirals, because they are then rerun more often, increasing the load further.** (d) Three things. **Clearly identify what the failure is: the worst case is an error that just says “assertion failed” and a stack trace,** where a good one anticipates the runner's unfamiliarity and gives context. **Minimise the effort of identifying the root cause, since a stack trace is not useful when the call chain spans process boundaries** — produce a trace across that chain, or automation that narrows the culprit. And **provide support and contact information.** On ownership: **larger tests must have documented owners who can review changes and be counted on for support when the test fails;** without one, **the test is harder to update, failures take longer to resolve — and the test rots.** Integration tests belong to the **project lead** and feature tests to a **feature owner — empowered, able to maintain it, and incentivised to.**

Spaced-review tracker

Review at **day 1, 3, 7, 16, 35**. Write the date in the box when you pass. On a fail, reset that row to a 1-day interval and start again.

ITEM	DAY 1	DAY 3	DAY 7	DAY 16	DAY 35	FAILS
1 • The four elements of the simplest test						
2 • The two purposes, and which one scales with change						
3 • Why a bad suite is worse than no suite						
4 • Size — the three levels and their constraints						
5 • Scope — and the two cases where it comes apart from size						
6 • The mix, and the two antipatterns with their causes						
7 • Hermetic, obvious on inspection, and no control flow						
8 • The flakiness arithmetic and the 1% threshold						
9 • Test everything you don't want to break						
10 • Why coverage is weak evidence, twice over						
11 • Brittle and slow — how a big suite turns on you						
12 • Exploratory testing, and the step after it						
13 • Maintainability, and its two failure categories						
14 • Brittle versus flaky						
15 • The four kinds of change, and the expensive one						
16 • Test via public APIs — the argument and the definition						
17 • The three rules of thumb for what counts as a unit						
18 • State versus interactions, and the two failure directions						
19 • Complete and concise						
20 • Behaviours not methods, and given/when/then						
21 • Naming after the behaviour, and the “and” test						
22 • No logic in tests, and why						
23 • What a good failure message carries						
24 • Descriptive over deduplicated, and the three sharing patterns						
25 • Seams, and why testability is an upfront investment						
26 • The three techniques, and the three impacts						
27 • When a real implementation is preferred — and the three tests						
28 • Fidelity of a fake — to the contract, from the test's view						
29 • Who owns a fake, where it belongs, and why it needs tests						

30 • The three dangers of overused stubbing, and the tell						
31 • When stubbing is right, and its two constraints						
32 • Change-detector tests, and the two warranted cases						
33 • State-changing versus non-state-changing calls						
34 • Fidelity, and why more of it is not always right						
35 • The five gaps unit tests leave						
36 • The three developer-friendly properties, plus ownership and standardisation						
37 • How low fidelity compounds, and why smaller is still better						
38 • The four phases, and hermeticity versus fidelity						
39 • Seeded data and traffic; the three sources; the three verifications						
40 • Keeping larger tests fast, unflaky, understandable and owned						

Final quiz — no notes, twenty minutes, write the answers

1. Give the four components of the most basic automated check, name both reasons for having such checks at all, and state where a body of them gets its worth and what follows when that worth is lost.

.....

2. Define the two independent ways a check is categorised, give the three grades of each, and supply one example in each direction where the two grades diverge.

.....

3. State the recommended proportions and the reasoning behind that shape, name both malformed distributions with what produces each, and say what the broadest checks should not be counted on to do.

.....

4. Work through the sums that make sporadic failures expensive, give the proportion at which people stop believing the results, explain why automatic repetition is not a cure, and say why branching and repetition are barred from check code.

.....

5. List the four categories of modification to production code, say which one ought to require editing existing checks and why that one costs more, and state the two possible conclusions when a restructuring breaks something.

.....

6. Explain why exercising a component the way its consumers do is the strongest defence against fragility, define what counts as a consumer-facing surface here, and give the three heuristics for deciding what constitutes a testable component.

.....

7. Contrast checking the resulting condition of a system with checking the sequence of calls it made, give both ways the latter misleads, and name the two circumstances that justify it plus the category of function it should be limited to.

8. Rank the four options for a dependency from most to least faithful, say what a passing result licenses you to assert at each, and give the three requirements that make a lightweight stand-in trustworthy.

9. Name the three harms caused by leaning too heavily on hardcoded return values, give the symptom that reveals you have gone too far, and state the situation where the practice is nonetheless the correct one.

10. Enumerate the five blind spots of narrow checks, name the three qualities that broader ones sacrifice along with the two organisational difficulties they add, and give the pair of properties of a stood-up environment that pull against each other.
